

DATA SCIENCE WITH PYTHON

Why Python is Important for Data Science?

1. **Easy to Learn:** Being an open-source platform, Python has a simple and intuitive syntax that is easy to learn and read. This makes it a great language for beginners to learn data science.
2. **Cross-Platform:** Being a developer, you don't need to worry about the data types. The reason is, Python allows developers to run the code on Windows, Mac OS X, UNIX, and Linux.
3. **Portable:** Being an easy & beginner's friendly programming language, Python is highly portable in nature which means that a developer can run their code on different machines without making any further changes.
4. **Extensive Library:** Python has several powerful libraries that make data analysis and visualization easy. Pandas is a library for data manipulation and analysis, NumPy is a library for numerical computation, and Matplotlib is a library for data visualization.
5. **Community Support:** Python has a large and active community that supports and contributes to the development of various libraries and tools for data science. This community has created many useful libraries, including Pandas, NumPy, matplotlib, and SciPy, which are widely used in data science.

Python For Data Science Benefits:

1. Data Analysis
2. Data Visualizations
3. Machine Learning
4. Deep Learning
5. Image processing
6. Computer Vision
7. Natural Language Processing (NLP)

Data Analysis with python:

Data analysis is a technique of collecting, transforming and organizing data to make future prediction and informed data-driven decisions. It's about turning raw data into valuable insights that can be used to solve problems.

Process of data analysis:

1. **Defining the Objective:** Before analyzing data, you must know what you're looking for. This step involves asking the right questions and defining the specific problem you're trying to solve.
2. **Data Collection:** Gather the raw data from various sources, such as databases, surveys, or external websites.

DATA SCIENCE WITH PYTHON

3. Data Cleaning: This is often the most time-consuming part of the process. Raw data is messy, so it needs to be cleaned by handling missing values, removing duplicates, and correcting errors to ensure its quality and accuracy.

4. Exploratory Data Analysis (EDA): Explore the cleaned data using visualizations and statistical summaries to identify patterns, trends, and anomalies. This helps you understand the data's characteristics.

5. Modelling and Analysis: This is where you apply statistical and logical techniques to find answers to your questions. This can range from simple statistical calculations to building complex machine learning models.

6. Interpretation and Visualization: Interpret the results of analysis and present findings in a clear, often using charts, graphs, and dashboards to help stakeholders understand the insights.

Python Packages Used in Data Analysis:

1. NumPy (Fast numerical operations, arrays, math functions)
2. Pandas (Tabular data manipulation (DataFrames))
3. Matplotlib (Basic plotting (line, bar, scatter, etc.))
4. Seaborn (Statistical plots with beautiful defaults)
5. Scikit-learn (Machine learning algorithms and pipelines)
6. SciPy (Scientific computing and statistical tools)

Data Analysis with NumPy(Numerical Computing):

NumPy is the fundamental Python library for numerical computing. It's the core package for data analysis and scientific computing, providing a high-performance ndarray (n-dimensional array) object. This array object allows for efficient storage and manipulation of large datasets, which is essential for data science tasks.

- **ndarray object** – Stores homogeneous data in n-dimensional arrays for fast processing.
- NumPy arrays are homogeneous, meaning all elements must be the same type, allowing efficient computation.
- Vectorized operations in NumPy can be 10 to 100 times faster than equivalent Python loops.
- **Importing NumPy** – `import numpy as np`.

DATA SCIENCE WITH PYTHON

- **Why need NumPy:**

1. NumPy speeds up math operations like addition and multiplication on large groups of numbers compared to regular Python.
2. It's good for handling large lists of numbers (arrays), so you don't have to write complicated loops.
3. It gives ready-to-use functions for statistics, algebra and random numbers.
4. Libraries like Pandas, SciPy, TensorFlow and many others are built on top of NumPy.
5. NumPy uses less memory and stores data more efficiently, which matters when working with lots of data.

- **NumPy Array:**

NumPy arrays (ndarrays) are the backbone of the library. This section covers how to create and manipulate arrays effectively for data storage and processing.

```
import numpy as np

my_list = [1, 2, 3, 4, 5]

numpy_array = np.array(my_list)

print("Simple NumPy Array:", numpy_array)
```

output: [1 2 3 4 5]

Initialize a Python NumPy Array Using Special Functions

NumPy provides several built-in functions to generate arrays with specific properties.

- [np.zeros\(\)](#): Creates an array filled with zeros.
- [np.ones\(\)](#): Creates an array filled with ones.
- [np.arange\(\)](#): Creates an array with values that are evenly spaced within a given range.
- [np.linspace\(\)](#): Creates an array with values that are evenly spaced over a specified interval.

```
import numpy as np

zeros_array = np.zeros((2, 3))

ones_array = np.ones((3, 3))

range_array = np.arange(0, 10, 2) # start, stop, step

linspace_array = np.linspace(0, 1, 5) # start, stop, num

print("Zero Array:", "\n", zeros_array)

print("Ones Array:", "\n", ones_array)

print("Range Array:", "\n", range_array)

print("Linspace Array:", "\n", linspace_array)
```

DATA SCIENCE WITH PYTHON

output:

```
Zero Array
[[0. 0. 0.]
 [0. 0. 0.]]
Zero Array
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
Range Array
[0 2 4 6 8]
Linspace Array
[0.  0.25 0.5  0.75 1.  ]
```

Create Python Numpy Arrays Using Random Number Generation

- [np.random.rand\(\)](#): Creates an array of specified shape and fills it with random values sampled from a uniform distribution over [0, 1).
- [np.random.randn\(\)](#): Creates an array of specified shape and fills it with random values sampled from a standard normal distribution.
- [np.random.randint\(\)](#): Creates an array of specified shape and fills it with random integers within a given range.

```
import numpy as np

random_array = np.random.rand(2, 3)

randint_array = np.random.randint(1, 10, size=(2, 3))

print(random_array)

print(randint_array)
```

```
output: [[0.87948864 0.55022063 0.29237533]
 [0.99475413 0.76666244 0.55240304]]

[[6 6 3]
 [8 5 8]]
```

Numpy Array Indexing:

Array indexing in NumPy refers to the method of accessing specific elements or subsets of data within an array. This feature allows us to retrieve, modify and manipulate data at specific positions or ranges helps in making it easier to work with large datasets.

DATA SCIENCE WITH PYTHON

1. Accessing Elements in 1D Arrays:

A 1D NumPy array is a sequence of values with positions called indices which starts at 0. We access elements by using these indices in square brackets like `arr[0]` for the first element. Negative indices count from the end so `arr[-1]` gives the last element.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(arr[0])

output :

10
```

2. Accessing Elements in Multidimensional Arrays:

In this we will see how to access elements in both 2D and 3D arrays using specific indices.

2D Arrays: We can access elements by specifying both row and column indices like `matrix[row, column]`.

```
import numpy as np

matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print(matrix[1, 2])

output:

6
```

3D Arrays: It can be visualized as a stack of 2D arrays, we need three indices-

1. **Depth:** Specifies the 2D slice.
2. **Row:** Specifies the row within the slice.
3. **Column:** Specifies the column within the row.

```
import numpy as np

cube = np.array([[[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]],
                [[10, 11, 12],
                 [13, 14, 15],
                 [16, 17, 18]]])

print(cube[1, 2, 0])

output: 16
```

DATA SCIENCE WITH PYTHON

NumPy Array Slicing

It allows us to extract a range of elements using the format **start:stop:step**. This can be done for both 1D and multidimensional arrays which allows us to select ranges of elements or submatrices easily.

Slicing 1D Arrays: For a 1D array, slicing returns a subset of elements between the **start** and **stop** indices.

```
import numpy as np  
arr = np.array([0, 1, 2, 3, 4, 5])  
print(arr[1:4])
```

Output:

```
[1 2 3]
```

Slicing Multidimensional Arrays: In this slicing can be applied to each dimension separately which allows us to extract submatrices or smaller blocks of data.

```
import numpy as np  
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
print(matrix[0:2, 1:3])
```

Output:

```
[ [2 3]  
  [5 6] ]
```

- **Mathematical Operations in NumPy:**

This section covers essential mathematical functions for array computations, including basic arithmetic, aggregation and mathematical transformations

Arithmetic operations are used for numerical computation and we can perform them on arrays using NumPy. With NumPy we can quickly add, subtract, multiply, divide and get power of elements in an array. NumPy performs these operations even with large amounts of data. In this article, we'll see at the basic arithmetic functions in NumPy and show how to use them for simple calculations.

1. Addition of Arrays

Addition is an arithmetic operation where the corresponding elements of two arrays are added together. In [NumPy](#) the addition of two arrays is done using the [np.add\(\)](#) function.

2. Subtraction of Arrays

We can subtract two arrays element-wise using the [np.subtract\(\)](#) function. This function subtracts each element of the second array from the corresponding element in the first array.

DATA SCIENCE WITH PYTHON

3. Multiplication of Arrays

Multiplication in NumPy can be done element-wise using the [`np.multiply\(\)`](#) function. This multiplies corresponding elements of two arrays.

4. Division of Arrays

Division is another important operation that is performed element-wise using the [`np.divide\(\)`](#) function. This divides each element of the first array by the corresponding element in the second array.

5. Exponentiation (Power)

It allows us to raise each element in an array to a specified power. In NumPy, this can be done using the [`np.power\(\)`](#) function.

6. Modulus Operation

It finds the remainder when one number is divided by another. In NumPy, you can use the [`np.mod\(\)`](#) function to calculate the modulus element-wise between two arrays.

```
import numpy as np

a = np.array([5, 72, 13, 100])
b = np.array([2, 5, 10, 30])

add_ans = np.add(a, b) #Addition
print(add_ans)

sub_ans = np.subtract(a, b) #Subtraction
print(sub_ans)

mul_ans = np.multiply(a, b) #Multiplication
print(mul_ans)

div_ans = np.divide(a, b) #Division
print(div_ans)

pow_ans = np.power(a, b) #Exponential
print(pow_ans)

mod_ans = np.mod(a, b) #Modulusw
print(mod_ans)
```

output:

DATA SCIENCE WITH PYTHON

```
[ 7 77 23 130]
```

```
[ 3 67 3 70]
```

```
[ 10 360 130 3000]
```

```
[ 2.5 14.4 1.3 3.33333333]
```

```
[25 1934917632 137858491849 1152921504606846976]
```

```
[ 1 2 3 10]
```

- **Linear Algebra in NumPy:**

NumPy provides built-in functions for linear algebra operations essential for scientific computing and machine learning applications.

Matrix manipulation in Python

Operation on Matrix :

1. `add()` :- This function is used to perform element wise matrix multiplication
2. `subtract()` :- This function is used to perform element wise matrix subtraction.
3. `divide()` :- This function is used to perform element wise matrix division.
4. `multiply()` :- This function is used to perform element wise matrix multiplication.
6. `sqrt()` :- This function is used to compute the square root of each element of matrix.
7. `"T"` :- This argument is used to transpose the specified matrix.

1. # Python code to demonstrate matrix operations

add(), subtract() and divide()

```
import numpy
```

```
x = numpy.array([[1, 2], [4, 5]])
```

```
y = numpy.array([[7, 8], [9, 10]])
```

using add() to add matrices

```
print ("The element wise addition of matrix is : ")
```

```
print (numpy.add(x,y))
```

using subtract() to subtract matrices

```
print ("The element wise subtraction of matrix is : ")
```

```
print (numpy.subtract(x,y))
```


DATA SCIENCE WITH PYTHON

using divide() to divide matrices

```
print ("The element wise division of matrix is : ")  
print (numpy.divide(x,y))
```

Output: The element wise addition of matrix is :

```
[[ 8 10]  
 [13 15]]
```

The element wise subtraction of matrix is :

```
[[ -6 -6]  
 [ -5 -5]]
```

The element wise division of matrix is :

```
[[0.14285714 0.25   ]  
 [0.44444444 0.5    ]]
```

2. # Python code to demonstrate matrix operations

multiply()

```
import numpy  
x = numpy.array([[1, 2], [4, 5]])  
y = numpy.array([[7, 8], [9, 10]])
```

using multiply() to multiply matrices element wise

```
print ("The element wise multiplication of matrix is : ")  
print (numpy.multiply(x,y))
```

Output: The element wise multiplication of matrix is :

```
[[ 7 16]  
 [36 50]]
```

3. # Python code to demonstrate matrix operations

sqrt(), and "T"

```
import numpy  
x = numpy.array([[1, 2], [4, 5]])  
y = numpy.array([[7, 8], [9, 10]])
```

using sqrt() to print the square root of matrix

DATA SCIENCE WITH PYTHON

```
print ("The element wise square root is : ")  
print (numpy.sqrt(x))  
  
# using "T" to transpose the matrix  
print ("The transpose of given matrix is : ")  
print (x.T)
```

Output: The element wise square root is :

```
[[1.    1.41421356]  
 [2.    2.23606798]]  
  
The transpose of given matrix is :  
[[1 4]  
 [2 5]]
```

- **Statistics in NumPy:**

NumPy provides a comprehensive set of functions for performing statistical analysis on arrays. These functions are essential for understanding the central tendency, spread, and distribution of data.

Measures of Central Tendency:

- np.mean(): Calculates the arithmetic mean (average) of array elements.
- np.median(): Calculates the median (middle value) of array elements.
- numpy.average(): Calculates the weighted average of array elements.

Measures of Dispersion (Spread):

- np.var(): Calculates the variance of array elements.
- np.std(): Calculates the standard deviation of array elements.
- np.ptp(): Calculates the peak-to-peak range (difference between maximum and minimum) of array elements.

Extremes:

- np.min(): Returns the minimum value of array elements.
- np.max(): Returns the maximum value of array elements.

Other Useful Statistical Functions:

- numpy.sum(): Calculates the sum of array elements
- numpy.prod(): Calculates the product of array elements.

DATA SCIENCE WITH PYTHON

Quantiles:

- `np.percentile()`: Calculates the nth percentile of array elements.

Note: NumPy doesn't include a built-in **`mode()`** function. For that, we use **`scipy.stats.mode()`** which returns the most frequent value in the array.

Why NumPy Doesn't Include `mode()`:

- **NumPy focuses on numerical operations:** It's optimized for fast, low-level array computations like arithmetic, linear algebra, and statistical summaries (mean, median, std). Mode, however, is more of a **frequency-based** operation, which isn't purely numerical—it's more categorical in nature.
- **Scipy handles advanced stats:** `scipy.stats.mode()` is part of SciPy's statistics module, which is designed to handle more complex or specialized statistical tasks—including mode calculation with support for multi-dimensional arrays and edge cases.

```
import numpy as np

# Create a sample NumPy array
data = np.array([1, 5, 7, 8, 9, 12, 14, 2, 6, 10])
print(f"Data Array: {data}")
print("-" * 30)

# Calculate mean
mean_value = np.mean(data)
print(f"Mean (np.mean): {mean_value}")

# Calculate median
median_value = np.median(data)
print(f"Median (np.median): {median_value}")

# Calculate standard deviation
std_dev = np.std(data)
print(f"Standard Deviation (np.std): {std_dev}")

# Find maximum and minimum
max_value = np.max(data)
min_value = np.min(data)
print(f"Maximum (np.max): {max_value}, Minimum (np.min): {min_value}")

# Calculate the 75th percentile
percentile_75 = np.percentile(data, 75)
```

DATA SCIENCE WITH PYTHON

```
print(f"75th Percentile (np.percentile): {percentile_75}")

print("-" * 30)

# 1. np.average (Weighted Average)

# np.average is similar to np.mean, but allows specifying weights.

# Without weights, it gives the same result as np.mean.

average_value = np.average(data)

print(f"Average (np.average): {average_value}")

# Example with weights: making the first 5 elements twice as important

weights = np.array([2, 2, 2, 2, 2, 1, 1, 1, 1, 1])

weighted_average = np.average(data, weights=weights)

print(f"Weighted Average: {weighted_average:.2f}")

# 2. np.var (Variance)

variance_value = np.var(data)

print(f"Variance (np.var): {variance_value:.2f}")

# 3. np.ptp (Peak-to-Peak Range)

range_value = np.ptp(data)

print(f"Peak-to-Peak Range (np.ptp): {range_value}") # 14 - 1 = 13

# 4. np.sum (Summation)

sum_value = np.sum(data)

print(f"Sum (np.sum): {sum_value}")

# 5. np.prod (Product)

product_value = np.prod(data)

print(f"Product (np.prod): {product_value}")
```

Output:

```
Data Array: [ 1 5 7 8 9 12 14 2 6 10] -----
Mean (np.mean): 7.4
Median (np.median): 7.5
Standard Deviation (np.std): 3.9924930606117284
Maximum (np.max): 14
Minimum (np.min): 1
75th Percentile (np.percentile): 9.75
```

DATA SCIENCE WITH PYTHON

Average (np.average): 7.4

Weighted Average: 6.93

Variance (np.var): 15.94

Peak-to-Peak Range (np.ptp): 13

Sum (np.sum): 74

Product (np.prod): 304819200

Data Analysis with Pandas(Data Manipulation)

Pandas is open-source Python library which is used for data manipulation and analysis. It consists of data structures and functions to perform efficient operations on data. It is well-suited for working with **tabular data** such as **spreadsheets** or **SQL tables**. It is used in data science because it works well with other important libraries. **It is built on top of the NumPy library** as it makes easier to manipulate and analyze. Pandas is used in other libraries such as:

- **Matplotlib** for plotting graphs
- **SciPy** for statistical analysis
- **Scikit-learn** for machine learning algorithms.
- **Key Data Structures:**

Pandas is built around two primary data structures:

1. **DataFrame:** This is the most frequently used structure. It's a **two-dimensional, tabular data structure** with labeled rows and columns, similar to a spreadsheet or a table in a relational database. It is ideal for storing datasets where columns can contain different types of data (e.g., text, numbers, dates).
 2. **Series:** This is a **one-dimensional labeled array**. It as a single column of a DataFrame or a list with an index.
- **Installing Pandas:** `""pip install pandas""`
 - **Import Pandas:** `""import pandas as pd""`
 - **Why need pandas:**
 1. Reading and writing data from various file formats like CSV, Excel and SQL databases.
 2. Cleaning and preparing data by handling missing values and filtering entries.
 3. Merging and joining multiple datasets seamlessly.
 4. Reshaping data through pivoting and stacking operations.

DATA SCIENCE WITH PYTHON

5. Conducting statistical analysis and generating descriptive statistics.

6. Visualizing data with integrated plotting capabilities.

- **Pandas Series:** Is a one-dimensional labeled array, capable of holding any data type (integers, strings, floating point numbers, Python objects, etc.). It as a single column in a spreadsheet or a Pandas DataFrame.

1. **Creating a Series from a Python List:** The simplest way is to pass a list or NumPy array to the `pd.Series()` constructor. Pandas automatically creates a numerical index for the labels (starting from 0).

```
import pandas as pd

import numpy as np

sales = [1500, 2200, 1850, 2900, 3100]

# Create a Series

sales_series = pd.Series(sales)

print("Sales Series:")

print(sales_series)
```

Output:

```
Sales Series:
0      1500
1      2200
2      1850
3      2900
4      3100
```

2. **Creating a Series with Custom Labels (Index):**

```
temps = [25.5, 27.1, 23.8, 28.0]

# Custom index labels (days of the week)

days = ['Mon', 'Tue', 'Wed', 'Thu']

# Create a Series with custom labels

temp_series = pd.Series(temps, index=days)

print("Temperature Series:")

print(temp_series)
```

DATA SCIENCE WITH PYTHON

Output:

Temperature Series:

Mon 25.5

Tue 27.1

Wed 23.8

Thu 28.0

3. Creating a Series from a NumPy Array:

```
import pandas as pd
import numpy as np
data = np.array(['p', 'y', 't', 'h', 'o', 'n'])
numpy_series = pd.Series(data)
print(numpy_series)
```

Output:

0	p
1	y
2	t
3	h
4	o
5	n

4. Creating a Series from a List:

```
import pandas as pd
data_list = ['p', 'y', 't', 'h', 'o', 'n']
list_series = pd.Series(data_list)
print(list_series)
```

Output:

0	p
1	y
2	t
3	h
4	o
5	n

DATA SCIENCE WITH PYTHON

5. Creating a series from a dictionary:

```
import pandas as pd

data_dict = {'p':0, 'y':1, 't':2, 'h':3, 'o':4, 'n':5}

dict_series = pd.Series(data_dict)

print(dict_series)
```

Output:

```
p      0
y      1
t      2
h      3
o      4
n      5
```

- **Pandas DataFrame:**

Pandas DataFrame is a two-dimensional data structure with labeled axes (rows and columns). It is created by loading the datasets from existing storage which can be a SQL database, a CSV file or an Excel file. It can be created from lists, dictionaries, a list of dictionaries etc.

```
import pandas as pd

import numpy as np

employee_data = {

    'Employee_ID': [101, 102, 103, 104, 105],

    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],

}

# Create the DataFrame

df = pd.DataFrame(employee_data)

print("Original DataFrame:") print(df)
```

Output:

Original DataFrame:

	Employee_ID	Name
0	101	Alice
1	102	Bob
2	103	Charlie

DATA SCIENCE WITH PYTHON

3	104	David
4	105	Eve

1. Creating a DataFrame from a List:

```
import pandas as pd
import numpy as np

data = [
    ['New York', 12.0, 8.4],
    ['London', 11.5, 8.9],
]

# Create the DataFrame, specifying column names

df_list = pd.DataFrame(data, columns=['City', 'Avg_Temp_C', 'Population_M'])
print("DataFrame from List:")
print(df_list)
```

Output:

	City	Avg_Temp_C	Population
0	New York	12.0	8.4
1	London	11.5	8.9

2. Creating a DataFrame from a Dictionary:

```
import pandas as pd
import numpy as np

employee_data = {
    'Employee_ID': [101, 102, 103, 104, 105],
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
}

# Create the DataFrame

df = pd.DataFrame(employee_data)
print("Original DataFrame:") print(df)
```

Output:

	Employee_ID	Name
--	-------------	------

DATA SCIENCE WITH PYTHON

0	101	Alice
1	102	Bob
2	103	Charlie
3	104	David
4	105	Eve

3. Creating a DataFrame from a NumPy array:

```
Import pandas as pd
Import numpy as np

# Create a 2D NumPy array (e.g., 4 rows and 3 columns)
# This array represents [Height_cm, Weight_kg, Age]
data data_array = np.array([
    [180, 75, 25],
    [165, 60, 30],

    [175, 70, 45],
    [188, 85, 22]
])

# Create the DataFrame, specifying the NumPy array and column names
df_numpy = pd.DataFrame(
    data_array,
    columns=['Height_cm', 'Weight_kg', 'Age'],
    index=['A', 'B', 'C', 'D'] # Optional: to set custom row labels
)

print("\nDataFrame from NumPy Array:")
print(df_numpy)
```

Output:

```
DataFrame from NumPy Array:
      Height_cm  Weight_kg  Age
A           180         75   25
B           165         60   30
C           175         70   45
```

DATA SCIENCE WITH PYTHON

D

188

85

22

Note: *import numpy*

You need to import a package like NumPy to access the functions and data structures it provides (like the fast ndarray). Python doesn't load every library by default; you only load what you need.

As np

The *as np* part is a shortcut. Instead of having to type the full package name every time you want to use a NumPy function, you only have to type *np*.

Instead of writing...

`numpy.array([1, 2, 3])`

`numpy.mean(data)`

You can write...

`np.array([1, 2, 3])`

`np.mean(data)`

Note: The Python statement `df = pd.DataFrame()` means, creating an empty Pandas DataFrame object and assigning it to the variable name *df*.

- *pd*: This is the common alias for the Pandas library.
- *DataFrame()*: This is the constructor function provided by Pandas to create a new DataFrame.
- *df*: This is the variable name used to reference the newly created empty DataFrame object.

Using `df = pd.DataFrame()`, convert raw data into structured format which is DataFrame and labelling the data with columns and rows.

• **Pandas DataFrame Index:**

Index in pandas dataframe act as reference for each row in dataset, means using labels or positions to **locate rows and columns** in a DataFrame. It can be numeric or based on specific column values.

1. Accessing and Modifying the Index:

```
import pandas as pd
```

```
data = {'Name': ['John', 'Alice', 'Bob', 'Eve', 'Charlie'],
```

```
       'Age': [25, 30, 22, 35, 28],
```

```
       'Gender': ['Male', 'Female', 'Male', 'Female', 'Male'],
```

```
       'Salary': [50000, 55000, 40000, 70000, 48000]}
```

```
df = pd.DataFrame(data)
```

```
print(df.index) # Accessing the index
```

output:

DATA SCIENCE WITH PYTHON

```
RangeIndex(start=0, stop=5, step=1)
```

2. Setting a Custom Index:

```
import pandas as pd

data = {'Name': ['John', 'Alice', 'Bob', 'Eve', 'Charlie'],
        'Age': [25, 30, 22, 35, 28],
        'Gender': ['Male', 'Female', 'Male', 'Female', 'Male'],
        'Salary': [50000, 55000, 40000, 70000, 48000]}

# Set 'Name' column as the index

df_with_index = df.set_index('Name')
print(df_with_index)
```

Output:

	Age	Gender	Salary
Name			
John	25	Male	50000
Alice	30	Female	55000
Bob	22	Male	40000
Eve	35	Female	70000
Charlie	28	Male	48000

3. Indexing with loc: The loc[] method in pandas allows to access rows and columns of a DataFrame using their labels, making it easy to retrieve specific data points.

```
import pandas as pd

data = {'Name': ['John', 'Alice', 'Bob', 'Eve', 'Charlie'],
        'Age': [25, 30, 22, 35, 28],
        'Gender': ['Male', 'Female', 'Male', 'Female', 'Male'],
        'Salary': [50000, 55000, 40000, 70000, 48000]}

}

row = df.loc['Alice']

print(row)
```

Output:

Age	30
-----	----

DATA SCIENCE WITH PYTHON

```
Gender    Female
Salary    55000
Name: Alice, dtype: object
```

- **Pandas Access Dataframe:**

Accessing a dataframe in pandas involves retrieving, exploring, and manipulating data stored within this structure. The most basic form of accessing a DataFrame is simply referring to it by its variable name.

```
import pandas as pd

data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 30, 22]}

df = pd.DataFrame(data)

# Display the entire DataFrame

print(df)
```

Output:

```
   Name  Age
0  John   25
1  Alice  30
2   Bob   22
```

1. Accessing Columns From DataFrame:

```
import pandas as pd

data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 30, 22]}

age_column = df['Age']

print(age_column)
```

Output:

```
0    25
1    30
2    22
3    35
```

DATA SCIENCE WITH PYTHON

4 28

2. **Accessing Rows From DataFrame:** To access specific rows in a DataFrame, use `iloc` (for positional indexing) or `loc` (for label-based indexing). These methods allow to retrieve rows based on their index positions or labels.

```
import pandas as pd

data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 30, 22]}

# Access the row at index 1 (second row)
second_row = df.iloc[1]

print(second_row)
```

Output:

Name	Alice
Age	30

3. **Accessing Rows Based on Conditions:**

```
import pandas as pd

data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 30, 22]}

# Access rows where 'Age' is greater than 25
filtered_data = df[df['Age'] > 25]

print(filtered_data)
```

Output:

	Name	Age
0	John	25
1	Alice	30

.loc : `df.loc[]` selects data by row and column labels. It supports single labels, lists, slices, and boolean masks.

.iloc : `df.iloc[]` selects data by row and column positions (zero-based). It supports integers, lists, slices, and boolean arrays.

DATA SCIENCE WITH PYTHON

- **Indexing and Selecting Data Frame:**

The **[] operator** is the basic and frequently used method for selecting in Pandas. It allows us to select columns and filter rows based on conditions. This method can be used to select individual columns or multiple columns.

- **Indexing Data using the [] Operator:** The **[] operator** is the basic and frequently used method for indexing in Pandas. It allows us to select columns and filter rows based on conditions, means using labels or positions to **locate rows and columns** in a DataFrame.

1. **Selecting a Single Column:**

```
import pandas as pd

# Sample DataFrame

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [25, 30, 35],
    "City": ["New York", "London", "Paris"]
}

df = pd.DataFrame(data)

# Selecting a single column

ages = df["Age"]

print(ages)
```

Output:

```
0    25
1    30
2    35
```

2. **Selecting Multiple Columns:**

```
import pandas as pd

# Sample DataFrame

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [25, 30, 35],
    "City": ["New York", "London", "Paris"]
}
```

DATA SCIENCE WITH PYTHON

```
df = pd.DataFrame(data)

# Selecting multiple columns
subset = df[["Name", "City"]]

print(subset)
```

Output:

	Name	City
0	Alice	New York
1	Bob	London
2	Charlie	Paris

- **Indexing with .loc[]:** The .loc[] function is used for label-based indexing. It allows us to access rows and columns by their labels

1. Selecting a Row by Label:

```
import pandas as pd

# Sample DataFrame with player names as index
data = pd.DataFrame({
    "Team": ["Celtics", "Celtics", "Celtics"],
    "Points": [12, 5, 8]
}, index=["Avery Bradley", "R.J. Hunter", "Marcus Smart"])

# Selecting multiple rows by label
rows = data.loc["Avery Bradley", "R.J. Hunter"]

print(rows)
```

Output:

	Team	Points
Avery Bradley	Celtics	12
R.J. Hunter	Celtics	5

2. Selecting row and columns:

```
import pandas as pd

# Example DataFrame
data = pd.DataFrame({
    "Team": ["Celtics", "Celtics", "Celtics"],
```


DATA SCIENCE WITH PYTHON

```
"Number": [0, 28, 36],
"Position": ["PG", "SG", "PG"]
}, index=["Avery Bradley", "R.J. Hunter", "Marcus Smart"])

# Selecting specific rows AND specific columns

selection = data.loc[["Avery Bradley", "R.J. Hunter"], ["Team", "Number", "Position"]]
print(selection)
```

Output:

	Team	Number	Position
Avery Bradley	Celtics	0	PG
R.J. Hunter	Celtics	28	SG

- **Indexing with `.iloc[ ]`:** The `.iloc[ ]` function is used for position-based indexing. It allows us to access rows and columns by their integer positions. It is similar to `.loc[ ]` but only accepts integer-based indices to specify rows and columns.

1. Selecting Rows:

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie", "David"],
    "Age": [25, 30, 35, 40],
    "City": ["New York", "London", "Paris", "Berlin"]
}

df = pd.DataFrame(data)

# Select multiple rows by position

rows = df.iloc[[0, 2]] # first row (0) and third row (2)
print(rows)
```

Output:

	Name	Age	City
0	Alice	25	New York
2	Charlie	35	Paris

DATA SCIENCE WITH PYTHON

2. Selecting rows and column:

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie", "David"],
    "Age": [25, 30, 35, 40],
    "City": ["New York", "London", "Paris", "Berlin"]
}

df = pd.DataFrame(data)

# Select specific rows and columns
selection = df.iloc[[0, 2], [0, 2]] # rows 0 & 2, columns 0 & 2
print(selection)
```

Output:

	Name	City
0	Alice	New York
2	Charlie	Paris

- **Filtering Data Frame:**

Data frame filtering is the process of selecting a subset of rows and/or columns from a larger data frame based on specific conditions or criteria.

Key Concepts

- **Conditions:** These are logical expressions (e.g., greater than, equal to, containing a specific substring, or within a range) that determine which data points are included in the resulting subset.
- **Boolean Indexing/Masking:** In programming languages like Python with Pandas, filtering often involves creating a "mask" of True and False values, where True indicates that a row meets the condition and should be kept, and False indicates it should be excluded.

Common Accessors Summary (Cheat Sheet)

<u>Accessor</u>	<u>Used For</u>
.loc	Label-based filtering
.iloc	Position-based filtering
.query()	SQL-like filtering

DATA SCIENCE WITH PYTHON

.str	String operations
.dt	Date-time operations
.isin()	Membership filtering
.between()	Range filtering
.isna()	Missing values

Creating a sample data frame:

```
import pandas as pd

df = pd.DataFrame({'Name': [' RACHEL ', ' MONICA ', ' PHOEBE ',
                             ' ROSS ', 'CHANDLER', ' JOEY '],
                  'Age': [30, 35, 37, 33, 34, 30],
                  'Salary': [100000, 93000, 88000, 120000, 94000, 95000],
                  'JOB': ['DESIGNER', 'CHEF', 'MASUS', 'PALENTOLOGY',
                          'IT', 'ARTIST']})

display(dataFrame)
```

1. Row Filtering using Boolean Condition:

- **Single Condition:**

```
# Single condition

filtered_df = df [df ['Age'] > 30]

display(filtered_df)
```

Output:

	Name	Age	Salary	JOB
1	MONICA	35	93000	CHEF
2	PHOEBE	37	88000	MASUS
3	ROSS	33	120000	PALENTOLOGY
4	CHANDLER	34	94000	IT

- **Multiple Condition:**

```
# Selecting multiple column

filtered_df = df [ (df ['Age'] > 30) & ( df ['Salary'] > 100000) ]

print(filtered_df )
```

DATA SCIENCE WITH PYTHON

Output:

	Name	Age	Salary	JOB
3	ROSS	33	120000	PALENTOLOG

- **OR Conditions:**

OR condition

```
filtered_df = df [(df ['Age'] > 30) | (df ['JOB'] == 'DESIGNER')]  
print(filtered_df )
```

Output:

	Name	Age	Salary	JOB
0	RACHEL	30	100000	DESIGNER
1	MONICA	35	93000	CHEF
2	PHOEBE	37	88000	MASUS
3	ROSS	33	120000	PALENTOLOGY
4	CHANDLER	34	94000	IT

Here RACHEL(index 0) is included because the condition is Age is greater than 30 and JOB is DESIGNER..

- 2. .loc — Label-Based Filtering:** .loc is a label-based indexer used to select rows and columns in a DataFrame by their labels (names).

It's especially useful in filtering, because you can combine conditions with row/column labels to extract exactly the data you want.

- **Filter rows by condition:**

```
filtered_df = df.loc [df ['Age'] > 30]  
print(filtered_df )
```

Output:

	Name	Age	Salary	JOB
1	MONICA	35	93000	CHEF
2	PHOEBE	37	88000	MASUS
3	ROSS	33	120000	PALENTOLOGY
4	CHANDLER	34	94000	IT

DATA SCIENCE WITH PYTHON

- **Filter rows and columns:**

```
Filtered_df = df.loc [ df['Age'] > 30, ['Name', 'Salary']]  
Print(filtered_df)
```

Output

	Name	Salary
1	MONICA	93000
2	PHOEBE	88000
3	ROSS	120000
4	CHANDLER	94000

- **Filter by index labels:**

```
filtered_df = df.loc[0:5]  
print(filtered_df)
```

Output:

	Name	Age	Salary	JOB
0	RACHEL	30	100000	DESIGNER
1	MONICA	35	93000	CHEF
2	PHOEBE	37	88000	MASUS
3	ROSS	33	120000	PALENTOLOGY
4	CHANDLER	34	94000	IT
5	JOEY	30	95000	ARTIST

loc[0:5] uses **label-based slicing**.

Since the DataFrame has the default integer index (0, 1, 2, 3, 4, 5), .loc[0:5] will select **all rows from index 0 through 5 inclusive**.

That means it returns **all 6 rows** of your sample data.

3. **.iloc — Position-Based Filtering:** .iloc in Pandas is used for position-based indexing. It selects rows and columns only by their integer positions (like array indices), starting from 0.

- **Select Rows:**

```
filtered_df = df.iloc[0:5]  
print(filtered_df)
```

Output:

	Name	Age	Salary	JOB
--	------	-----	--------	-----

DATA SCIENCE WITH PYTHON

```
0  RACHEL  30 100000  DESIGNER
1  MONICA  35  93000   CHEF
2  PHOEBE  37  88000   MASUS
3   ROSS   33 120000  PALENTOLOGY
4 CHANDLER  34  94000   IT
```

`.iloc[0:5]` → position-based slicing.

It selects rows from position 0 up to 5 (exclusive).

So you'll get rows 0, 1, 2, 3, 4 (the first five rows).

Row at position 5 (JOEY) is excluded.

Key difference:

- `.iloc[start:end]` → **end is exclusive**. (Inclusive → the end value is included in the result.)
- `.loc[start:end]` → **end is inclusive**. (Exclusive → the end value is excluded from the result.)

• Select Rows and Columns:

```
filtered_df = df.iloc[0:5, 1:3]
print(filtered_df)
```

Output:

```
Age Salary
0  30 100000
1  35  93000
2  37  88000
3  33 120000
4  34  94000
```

`0:5` → row positions 0 through 4 (end exclusive). That means the first 5 rows.

`1:3` → column positions 1 through 2 (end exclusive). That means the 2nd and 3rd columns (since indexing starts at 0).

In DataFrame, columns are:

0 → "Name"

1 → "Age"

2 → "Salary"

DATA SCIENCE WITH PYTHON

3 → "JOB"

So you're selecting rows 0–4 and columns "Age" and "Salary".

4. .query() – SQL-like Filtering:

```
filtered_df = dataframe.query("Age > 30 and Salary < 100000")
print(filtered_df)
```

Output:

```
   Name Age Salary  JOB
1  MONICA 35  93000  CHEF
2  PHOEBE 37  88000 MASUS
4 CHANDLER 34  94000   IT
```

5. .isin() – Membership Filtering:

```
filtered_df = dataframe[dataframe['JOB'].isin(['IT', 'CHEF'])]
print(filtered_df)
```

Output:

```
   Name Age Salary  JOB
1  MONICA 35  93000  CHEF
4 CHANDLER 34  94000   IT
```

.isin(['IT', 'CHEF']) checks whether the value in the **JOB** column is either "IT" or "CHEF"

6. .str Accessor – String-Based Filtering: The .str accessor in Pandas is a special property that lets you apply string methods directly to each element of a Series (or DataFrame column) that contains text.

- Only works on columns (Series) with string or object dtype.
- Provides access to most Python string methods (lower, upper, strip, etc.) plus some extra Pandas-specific ones (contains, match, slice)

```
filtered_df = dataframe[dataframe['JOB'].str.startswith('A')]
print(filtered_df)
```

Output:

```
   Name Age Salary  JOB
5  JOEY 30  95000 ARTIST
```

Only "ARTIST" starts with "A".

So the filter will return the row for JOEY

DATA SCIENCE WITH PYTHON

7. **.between() – Numeric Range Filtering:** .between() is a method in Pandas used to check whether values in a Series (a column) fall between two boundaries. It returns a Boolean Series (True/False for each row), which you can use for filtering. By default, the boundaries are inclusive (meaning the start and end values are included).

```
filtered_df = dataframe[dataframe['Age'].between(30, 35)]  
print(filtered_df)
```

Output:

	Name	Age	Salary	JOB
0	RACHEL	30	100000	DESIGNER
1	MONICA	35	93000	CHEF
3	ROSS	33	120000	PALENTOLOGY
4	CHANDLER	34	94000	IT
5	JOEY	30	95000	ARTIST

.between(30, 35) → selects rows where Age is between 30 and 35 inclusive.

8. Missing Value Filtering:

- **Check missing values:**

```
filtered_df = df[df['Salary'].isna()]  
print(filtered_df)
```

Output:

```
Empty DataFrame  
Columns: [Name, Age, Salary, JOB]  
Index: []
```

So the result is empty because there are no NaN values in the Salary column.

- **Remove missing values:**

```
filtered_df = df.dropna()  
print(filtered_df)
```

output:

	Name	Age	Salary	JOB
0	RACHEL	30	100000	DESIGNER
1	MONICA	35	93000	CHEF

DATA SCIENCE WITH PYTHON

```
2  PHOEBE  37  88000    MASUS
3   ROSS   33 120000 PALENTOLOGY
4  CHANDLER 34  94000      IT
5   JOEY   30  95000    ARTIST
```

- `.dropna()` removes all rows that contain any missing values (NaN). In sample DataFrame, none of the columns (Name, Age, Salary, JOB) have missing values. Therefore, no rows are dropped. In short: since there are no NaN values, `.dropna()` doesn't remove anything, and the output is identical to the original DataFrame.

9. Date & Time Filtering — `.dt` Accessor:

```
import pandas as pd

df = pd.DataFrame({
    'date': pd.to_datetime([
        '2024-05-10', '2024-05-20',
        '2024-06-15', '2023-05-12'
    ]),
    'value': [10, 20, 30, 40]
})

filtered_df = df[(df['date'].dt.year == 2024) & (df['date'].dt.month == 5)]
print(filtered_df)print(filtered_df)
```

Output:

	date	value
0	2024-05-10	10
1	2024-05-20	20

NOTE: although in pandas Data Frame there are some other types of operations like Slicing, merge, join, concatenate, sort and pivot table

DATA SCIENCE WITH PYTHON

- **Data Input and Output (I/O):**

- A. Loading a CSV file into DATAFRAME :**

- **What is CSV file:** A CSV(Comma Separated Values) file is a plain text file that stores tabular data, where each record is on a new line and each field is separated by a comma. It's one of the simplest and most widely used formats for exchanging data between spreadsheets, databases, and programming languages.
 - Pandas provides the function `read_csv()` to load CSV files into a DataFrame.
 - **Bassic Syntax:**

```
import pandas as pd

df = pd.read_csv("employees.csv")

print(df)
```

Output:

	First Name	Age	Salary	Job
0	Rachel	30	100000	Designer
1	Monica	35	93000	Game Developer
2	Ross	33	120000	Server Maintenance
3	Joey	30	95000	Designer

- "employees.csv" is the file name or file path.
 - "read_csv()" is to load CSV files into a DataFrame.

- 1. Reading CSV from the Same Folder:**

```
import pandas as pd

df = pd.read_csv("employees.csv")

print(df.head())
```

Reads file from the current working directory

`head()` shows first 5 rows

- **Pandas read CSV syntax with its parameters:**
 - `sep`: It stands for separator, default is `,`.
 - `header`: It accepts int, a list of int, row numbers to use as the column names, and the start of the data. If no names are passed, i.e., `header=None`, then, it will display the first column as 0, the second as 1, and so on.
 - `usecols`: Retrieves only selected columns from the CSV file.
 - `nrows`: Number of rows to be displayed from the dataset.

DATA SCIENCE WITH PYTHON

- `index_col`: If None, there are no index numbers displayed along with records.
- `skiprows`: Skips passed rows in the new data frame.

- 2. Read specific columns using `usecols` in `read_csv`:** The `usecols` parameter allows to load only specific columns from a CSV file. This reduces memory usage and processing time by importing only the required data.

```
Import pandas as pd
```

```
df = pd.read_csv("employees.csv", usecols = ['First Name', ''])
```

```
print(df)
```

Output:

	First Name	Job
0	Rachel	Designer
1	Monica	Chef
2	Ross	Paleontology
3	Joey	Artist

here in `employees.csv` file just retrieve the 'First Name' and 'Job' column.

- 3. Setting an Index Column using `index_col`:** The `index_col` parameter sets one or more columns as the DataFrame index, making the specified column(s) act as row labels for easier data referencing.

```
Import pandas as pd
```

```
df = pd.read_csv("employees.csv", index_col="First Name")
```

```
print(df)
```

Output:

	Age	Salary	Job
First Name			
Rachel	30	100000	Designer
Monica	35	93000	Chef
Ross	33	120000	Paleontology
Joey	30	95000	Artist

- 4. Handling Missing Values Using `read_csv`:** The `na_values` parameter replaces specified strings (e.g., "N/A", "Unknown") with NaN, enabling consistent handling of missing or incomplete data during analysis.

```
Import pandas as pd
```

```
df = pd.read_csv("employees.csv", na_values=["N/A", "Unknown"])
```

DATA SCIENCE WITH PYTHON

```
print(df)
```

Output:

	First Name	Age	Salary	Job
0	Rachel	30	100000	Designer
1	Monica	35	NaN	Chef

5. **Using nrows in read_csv():** The nrows parameter limits the number of rows read from a file, enabling quick previews or partial data loading for large datasets. Here, we just display only 5 rows using nrows parameter.

```
Import pandas as pd

df = pd.read_csv("employees.csv", nrows=2)

print(df)
```

Output:

	First Name	Age	Salary	Job
0	Rachel	30	100000	Designer
1	Monica	35	93000	Chef

6. **Using skiprows in read_csv():** The skiprows parameter skips unnecessary rows at the start of a file, which is useful for ignoring metadata or extra headers that are not part of the dataset.

```
Import pandas as pd

df = pd.read_csv("employees.csv", skiprows=2) #Skip first two rows

print(df)
```

Output:

3	Ross	33	120000	Paleontology
4	Joey	30	95000	Artist

7. **Using header in read_csv():**

- The header parameter tells Pandas which row(s) in the CSV file should be used as column names.
- By default, header='infer', which means Pandas assumes the first row (row 0) contains the column names.
- If you set header=None, Pandas will not treat any row as column names and instead assign default numeric column labels (0, 1, 2, ...).
- You can also pass a list of row numbers (e.g., header=[0,1]) to create a MultiIndex (hierarchical column names).

```
Import pandas as pd

df = pd.read_csv("employees.csv", header=None)
```

DATA SCIENCE WITH PYTHON

```
print(df)
```

Output:

	0	1	2	3
0	First Name	Age	Salary	Job
1	Rachel	30	100000	Designer
2	Monica	35	93000	Chef
3	Ross	33	120000	Paleontology
4	Joey	30	95000	Artist

- 8. Using dtype in read_csv():** The dtype parameter in `pd.read_csv()` lets you specify the data type for one or more columns while reading a CSV file.

```
import pandas as pd

df = pd.read_csv("employees.csv", dtype={"Age": float})

print(df.dtypes)
```

Output:

```
First Name    object
Age           float64
Salary        int64
Job           object
dtype: object
```

- 9. Loading a CSV Data from a URL:** Pandas allows to directly read a CSV file hosted on the internet using the file's URL. This can be incredibly useful when working with datasets shared on websites, cloud storage, or public repositories like GitHub.

```
import pandas as pd

url = "https://people.sc.fsu.edu/~jburkardt/data/csv/hw_200.csv" # sample CSV file

df = pd.read_csv(url)

print(df.head())
```

Output:

	"Index"	"Height(Inches)"	"Weight(Pounds)"
0	1	65	112
1	2	71	220
2	3	69	185

DATA SCIENCE WITH PYTHON

3	4	68	110
4	5	67	120

10. Reading CSV Using File Path:

```
import pandas as pd

# Read CSV using full file path

df = pd.read_csv("C:/Users/Riktam/Documents/employees.csv")

print(df.head())
```

Output:

	First Name	Age	Salary	Job
0	Rachel	30	100000	Designer
1	Monica	35	93000	Chef
2	Ross	33	120000	Paleontology
3	Joey	30	95000	Artist

B. Loading Data from an Excel File: For importing an Excel file into Python using Pandas we have to use `pandas.read_excel()` function.

- `pd.read_excel("file_name.xlsx")` to load Excel files.
- sheet selection, headers, columns, rows, data types, and missing values just like with `read_csv()`.
- Pandas requires the `openpyxl` library for `.xlsx` files (install with `pip install openpyxl` if not already available).

```
import pandas as pd

# Read the Excel file

df = pd.read_excel("employees.xlsx")

print(df.head())
```

Output:

	First Name	Age	Salary	Job
0	Rachel	30	100000	Designer
1	Monica	35	93000	Chef
2	Ross	33	120000	Paleontology
3	Joey	30	95000	Artist

DATA SCIENCE WITH PYTHON

- **Key Parameters in read_excel:**

Parameter	Purpose	Example
sheet_name	Specify which sheet to read (default = first sheet).	sheet_name="Sheet1" sheet_name=[0,1]
header	Row number(s) to use as column names.	header = 0
Usecols	Select specific columns to read	usecols = ["First Name","Job"]
Nrows	Number of rows to read.	nrows = 10
skiprows	Skip given rows.	skiprows = [0,2]
dtype	Force column types.	dtype = {"Age": int}
na_values	Strings to treat as NaN.	na_values = ["NA","missing"]

C. Read JSON Files with Pandas: JSON (JavaScript Object Notation) store data using key-value pairs. Reading JSON files using Pandas is simple and helpful when working with data in .json format.

`pd.read_json()` is used to read JSON files and convert structured or nested JSON data into pandas DataFrames.

1. **Using `pd.read_json()` to Read JSON Files in Pandas:** The `pd.read_json()` function helps to read JSON data directly into a DataFrame. This method is used when working with standard JSON structures. If the file is located on a remote server we can also pass the URL instead of a local file path.

```
import pandas as pd
df = pd.read_json('data.json')
print(df.head())
```

2. **Reading JSON from File Path:** The `json_normalize()` is used in working with nested JSON structures. JSON from APIs often comes in nested form and this method helps to flatten it into a tabular format that's easier to work with in Pandas. This method is helpful when working with real-world JSON responses from APIs.

```
import pandas as pd
df = pd.read_json("C:/Users/HP/Desktop/data.json")
print(df.head())
```

DATA SCIENCE WITH PYTHON

3. Read JSON with Records Format:

```
[
    {"Name": "Rachel", "Age": 30, "Salary": 100000},
    {"Name": "Monica", "Age": 35, "Salary": 93000}
]

import pandas as pd

df = pd.read_json("data.json", orient="records")

print(df)
```

4. Read Nested JSON using `json_normalize()`:

A nested JSON is a JSON object that contains other JSON objects or arrays inside it. Instead of being flat (like a simple table), it has hierarchical structure.

Example of flat JSON:

```
[{"Name": "Rachel", "Age": 30, "Job": "Designer"},
 {"Name": "Monica", "Age": 35, "Job": "Chef"}]
```

Example of nested JSON:

```
[ {
    "Name": "Ross",
    "Age": 33,
    "Job": {"Title": "Paleontologist", "Department": "Science"}
},
{
    "Name": "Joey",
    "Age": 30,
    "Job": {"Title": "Actor", "Department": "Arts"}
}]
```


DATA SCIENCE WITH PYTHON

```
import json

import pandas as pd

# Sample nested JSON

data = '''

[ {

    "Name": "Ross",

    "Age": 33,

    "Job": {"Title": "Paleontologist", "Department": "Science"}

},

{

    "Name": "Joey",

    "Age": 30,

    "Job": {"Title": "Actor", "Department": "Arts"}

}] '''

# Load JSON into Python object

parsed = json.loads(data)

# Flatten nested JSON

df = pd.json_normalize(parsed)

print(df)
```

Output:

	Name	Age	Job.Title	Job.Department
0	Ross	33	Paleontologist	Science
1	Joey	30	Actor	Arts

5. Read JSON from URL (API Data):

```
import pandas as pd

url = "https://example.com/data.json"

df = pd.read_json(url)

print(df.head())
```

DATA SCIENCE WITH PYTHON

<u>Parameter</u>	<u>Purpose</u>	<u>Purpose</u>
<u>orient</u>	Defines JSON format (records, columns, index, etc.).	Orient = "records"
typ	Return type: frame (DataFrame) or series.	typ = "series"
Convert_dates	Convert date-like strings to datetime objects.	Convert_dates = True
encoding	Specify file encoding.	encoding = "utf-8"
lines	Read JSON file where each line is a separate JSON object.	Lines = True

Note: Pandas can also load data from SQL database, web scraping using BeautifulSoup...

D. Checking Missing Values in Pandas: Missing values in pandas can be detected using `isna()` / `isnull()`.

1. **Using `isnull()`:** `isnull()` returns a DataFrame of Boolean value where True represents missing data (NaN). This is simple if we want to find and fill missing data in a dataset.

```
import pandas as pd
import numpy as np

d = {'First Score': [100, 90, np.nan, 95],
     'Second Score': [30, 45, 56, np.nan],
     'Third Score': [np.nan, 40, 80, 98]}

df = pd.DataFrame(d)

mv = df.isnull()

print(mv)
```

Output:

```
First Score  Second Score  Third Score
0      False      False      True
1      False      False      False
2       True      False      False
3      False       True      False
```

DATA SCIENCE WITH PYTHON

Row 0 → "Third Score" is NaN → True.

Row 2 → "First Score" is NaN → True.

Row 3 → "Second Score" is NaN → True.

All other cells are valid → False.

2. **Using isna():** isna() returns a DataFrame of Boolean values where True indicates missing data (NaN). It is used to detect missing values just like isnull().

```
import pandas as pd
import numpy as np
data = {'Name': ['Amit', 'Sita', np.nan, 'Raj'],
        'Age': [25, np.nan, 22, 28]}
df = pd.DataFrame(data)
# Check for missing values using isna()
print(df.isna())
```

Output:

```
   Name  Age
0  False False
1  False  True
2   True False
3  False False
```

Row 0 → "Name" = Amit, "Age" = 25 → no missing → False False.

Row 1 → "Age" = NaN → True under Age.

Row 2 → "Name" = NaN → True under Name.

Row 3 → all values present → False False.

3. **Checking for Non-Missing Values Using notnull():** notnull() function returns a DataFrame with Boolean values where True indicates non-missing (valid) data. This function is useful when we want to focus only on the rows that have valid, non-missing values.

```
import pandas as pd
import numpy as np
d = {'First Score': [100, 90, np.nan, 95],
     'Second Score': [30, 45, 56, np.nan],
     'Third Score': [np.nan, 40, 80, 98]}
```

DATA SCIENCE WITH PYTHON

```
df = pd.DataFrame(d)
nmv = df.notnull()
print(nmv)
```

Output:

```
      First Score Second Score Third Score
0         True         True         False
1         True         True          True
2        False         True          True
3         True        False          True
```

E. Filling Missing Values in Pandas: These functions allow us to replace missing values with a specified value or use interpolation methods to find the missing data.

1. **Using fillna():** fillna() used to replace missing values (NaN) with a given value.

```
import pandas as pd
import numpy as np

d = {'First Score': [100, 90, np.nan, 95],
     'Second Score': [30, 45, 56, np.nan],
     'Third Score': [np.nan, 40, 80, 98]}

df = pd.DataFrame(d)

df.fillna(0)
```

Output:

```
      First Score Second Score Third Score
0         100.0         30.0          0.0
1          90.0         45.0         40.0
2           0.0         56.0         80.0
3          95.0           0.0         98.0
```

df.fillna(0) replaces all NaN values with 0.

Important: fillna() returns a new DataFrame by default. If you want to update df itself, you need df.fillna(0, inplace=True).

2. **Using replace():** The DataFrame.replace() method allows you to replace specific values in your DataFrame with new ones. Unlike fillna(), which only targets missing values, replace() can target any value (numbers, strings, NaN, etc.).

DATA SCIENCE WITH PYTHON

```
import pandas as pd

import numpy as np

d = {
    'First Score': [100, 90, np.nan, 95],
    'Second Score': [30, 45, 56, np.nan],
    'Third Score': [np.nan, 40, 80, 98]
}

df = pd.DataFrame(d)

# Replace NaN with a Value 0

df_replaced = df.replace(np.nan, 0)

print(df_replaced)
```

3. Difference between fillna() and replace():

- **fillna():**

Specialized for handling missing values (NaN) only.

Designed to fill gaps in your dataset.

Common strategies:

Replace NaN with a constant (df.fillna(0))

Replace NaN with column mean/median (df.fillna(df['col'].mean()))

Forward/Backward fill (df.fillna(method='ffill'), df.fillna(method='bfill'))

Example:

```
df.fillna(0) # replaces only NaN values with 0
```

- **replace():**

General-purpose replacement function.

Works on any value (not just NaN).

Can replace numbers, strings, regex patterns, or even dictionaries of mappings.

Example:

```
df.replace({90: 0, "Chef": "Cook"}) # replaces 90 with 0, "Chef" with "Cook"

df.replace(np.nan, 0) # can also replace NaN, but less specialized
```

DATA SCIENCE WITH PYTHON

F. Dropping Missing Values in Pandas: The `dropna()` function is used to remove rows or columns with NaN values. It can be used to drop data based on different conditions.

The main function is `DataFrame.dropna()`, which removes rows or columns containing NaN.

1. Drop Rows with Any Missing Values:

```
import pandas as pd
import numpy as np
d = {
    'First Score': [100, 90, np.nan, 95],
    'Second Score': [30, 45, 56, np.nan],
    'Third Score': [np.nan, 40, 80, 98]
}
df = pd.DataFrame(d)
df_drop = df.dropna()
print(df_drop)
```

Output:

	First Score	Second Score	Third Score
1	90.0	45.0	40.0

Row 0 → "Third Score" is NaN → dropped.

Row 2 → "First Score" is NaN → dropped.

Row 3 → "Second Score" is NaN → dropped.

Only Row 1 has no missing values, so it remains.

2. Drop Columns with Any Missing Values:

```
import pandas as pd
import numpy as np
d = {
    'First Score': [100, 90, np.nan, 95],
    'Second Score': [30, 45, 56, np.nan],
    'Third Score': [np.nan, 40, 80, 98]
}
df = pd.DataFrame(d)
```

DATA SCIENCE WITH PYTHON

```
df_drop_col = df.dropna(axis=1)
print(df_drop_col)
```

Output:

```
Empty DataFrame
Columns: []
Index: [0, 1, 2, 3]
```

When call `df.dropna(axis=1)`, Pandas interprets this as:

`axis=1` → operate on columns instead of rows.

`how='any'` (default) → drop a column if it contains any missing value (NaN).

Now, in your DataFrame:

"First Score" → contains a NaN at row 2.

"Second Score" → contains a NaN at row 3.

"Third Score" → contains a NaN at row 0.

Since every column has at least one NaN, Pandas drops all three columns.

The result is an empty DataFrame with no columns, but the index [0,1,2,3] is still present means the rows weren't dropped, only the columns removed.

3. Drop Rows Only if All Values Are Missing:

```
import pandas as pd
import numpy as np

d = {
    'First Score': [100, 90, np.nan, 95],

    'Second Score': [30, 45, 56, np.nan],
    'Third Score': [np.nan, 40, 80, 98]
}

df = pd.DataFrame(d)
df_drop_all = df.dropna(how='all')
print(df_drop_all)
```

Output:

DATA SCIENCE WITH PYTHON

	First Score	Second Score	Third Score
0	100.0	30.0	NaN
1	90.0	45.0	40.0
2	NaN	56.0	80.0
3	95.0	NaN	98.0

`dropna(how='all')` → drops rows only if all values in that row are NaN.

In DataFrame:

Row 0 → has values in "First Score" and "Second Score" → kept.

Row 1 → has values in all columns → kept.

Row 2 → has values in "Second Score" and "Third Score" → kept.

Row 3 → has values in "First Score" and "Third Score" → kept.

None of the rows are completely empty, so no rows are dropped

4. dropna() parameters:

- `dropna()` → Removes missing values (NaN) from a DataFrame or Series [`df.dropna()`]
- `axis=0` → Operates on rows (default) [`df.dropna(axis = 0)`]
- `axis=1` → Operates on columns [`df.dropna(axis = 1)`]
- `how='any'` → Drop if any value is NaN (default) [`df.dropna(how = any)`]
- `how='all'` → Drop if all values are NaN [`df.dropna(how = all)`]
- `thresh=n` → Keep rows/columns with at least n non-NaN values [`df.dropna(thresh = n)`]
- `inplace=True` → Modify the DataFrame directly (no new copy returned) [`df.dropna(inplace = True)`]

• **Pandas GroupBy:**

Pandas GroupBy is a method that lets you split your data into groups based on some criteria (like a column), apply functions (like sum, mean, count), and then combine the results back into a DataFrame. It's the foundation for summarizing, aggregating, and grouping data efficiently. It is based on the split–apply–combine strategy.

- Split → divide the dataset into groups based on one or more keys (e.g., a column like "Subject").
- Apply → perform an operation (aggregation, transformation, or filtration) on each group independently.
- Combine → merge the results back into a new DataFrame or Series.

DATA SCIENCE WITH PYTHON

1. **Summarizing:** Summarizing means creating a short, clear version of information that captures only the most important points. It reduces detail but keeps the core meaning. The goal is to make complex or large data easier to understand quickly.

- **Describe():** - Gives a statistical summary of numerical columns (count, mean, std, min, quartiles, max).

```
import pandas as pd

import numpy as np

df = pd.DataFrame({
    'Math': [90, 85, 88, 95],
    'Science': [80, 75, 92, 85]
})

print(df.describe())
```

Output:

	Math	Science
count	4.000000	4.000000
mean	89.500000	85.500000
std	4.203173	7.258644
min	85.000000	75.000000
25%	87.250000	81.250000
50%	89.000000	85.000000
75%	91.250000	89.250000
max	95.000000	92.000000

- **Info():** - Provides a concise summary of the DataFrame (rows, columns, data types, non-null counts, memory usage).

```
import pandas as pd

import numpy as np

df = pd.DataFrame({
    'Math': [90, 85, 88, 95],

    'Science': [80, 75, 92, 85]
})

print(df.info())
```

DATA SCIENCE WITH PYTHON

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Math    4 non-null       int64
1   Science  4 non-null       int64
dtypes: int64(2)
memory usage: 192.0 bytes
None
```

RangeIndex: 4 entries, 0 to 3 → shows row indices.

Data columns (total 2 columns) → shows number of columns.

Each column line shows:

Column name (Math, Science)

Non-null count (4 non-null)

Data type (int64)

dtypes: int64(2) → both columns are integers.

memory usage → small since only 4 rows.

None → because `print(df.info())` prints the summary and then returns None.

- **Unique():** returns the distinct values (Distinct values are the non-repeated, unique entries in a column.) present in a column (Series).

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'Math': [90, 85, 88, 95],
    'Science': [80, 75, 92, 85]
})

print(df['Math'].unique())
```

DATA SCIENCE WITH PYTHON

output:

```
[90 85 88 95]
```

df['Math'] selects the Math column as a Pandas Series.

.unique() finds all distinct values in the Series.

Since all values in Math are different (90, 85, 88, 95), the result is simply those values.

- **Columns:** It returns the list of column labels (names) in the DataFrame. This is a form of summarization because it gives you a quick overview of the dataset's structure — specifically, what fields (columns) exist.

```
import pandas as pd

df = pd.DataFrame({
    'Math': [90, 85, 88, 95],
    'Science': [80, 75, 92, 85]
})

print(df.columns)
```

Output:

```
Index(['Math', 'Science'], dtype='object')
```

This tells you the DataFrame has two columns: Math and Science.

2. **Aggregation:** Aggregation means applying a function to a group of values to reduce them into a single summary value. It's the "apply" step in the split-apply-combine process of Pandas. Common aggregation functions: sum(), mean(), count(), min(), max(), std(), median().

Aggregation Methods in Pandas:

sum() → total of values

mean() → average

count() → number of values

min() / max() → smallest / largest

std() → standard deviation

median() → middle value

agg() → apply multiple aggregation functions at once

DATA SCIENCE WITH PYTHON

```
import pandas as pd

df = pd.DataFrame({
    'Subject': ['Math', 'Math', 'Science', 'Science'],
    'Score': [90, 85, 95, 80]
})

# 1. sum()
print("Sum of Scores:")
print(df['Score'].sum(), "\n")

# 2. mean()
print("Mean of Scores:")
print(df['Score'].mean(), "\n")

# 3. median()
print("Median of Scores:")
print(df['Score'].median(), "\n")

# 4. min()
print("Minimum Score:")
print(df['Score'].min(), "\n")

# 5. max()
print("Maximum Score:")
print(df['Score'].max(), "\n")

# 6. count()
print("Count of Scores:")
print(df['Score'].count(), "\n")

# 7. std()
print("Standard Deviation of Scores:")
print(df['Score'].std(), "\n")

# 8. var()
print("Variance of Scores:")
print(df['Score'].var(), "\n")

# 9. agg() - multiple aggregations at once
print("Multiple Aggregations (mean, max, min, count):")
```

DATA SCIENCE WITH PYTHON

```
print(df['Score'].agg(['mean', 'max', 'min', 'count']), "\n")
```

Output:

Sum of Scores:

350

Mean of Scores:

87.5

Median of Scores:

87.5

Minimum Score:

80

Maximum Score:

95

Count of Scores:

4

Standard Deviation of Scores:

6.454972243679028

Variance of Scores:

41.666666666666664

Multiple Aggregations (mean, max, min, count):

mean 87.5

max 95.0

min 80.0

count 4.0

- 3. Groupby():** It splits DataFrame into groups based on one or more columns. Then can apply aggregation functions (like mean, sum, count) to each group separately. It's the split-apply-combine strategy:

Split → divide data into groups.

Apply → run aggregation on each group.

Combine → bring results back together.

```
import pandas as pd
```

```
df = pd.DataFrame({
```

```
    'Subject': ['Math', 'Math', 'Science', 'Science', 'English'],
```

DATA SCIENCE WITH PYTHON

```
'Score': [90, 85, 95, 80, 88],
'Student': ['A', 'B', 'C', 'D', 'E']
})

# 1. Group by Subject and calculate mean score

print("Mean Score per Subject:")
print(df.groupby('Subject')['Score'].mean(), "\n")

# 2. Group by Subject and calculate multiple aggregations

print("Multiple Aggregations per Subject:")
print(df.groupby('Subject')['Score'].agg(['mean', 'max', 'min', 'count']), "\n")

# 3. Group by Subject and Student, then sum scores

print("Group by Subject and Student (sum of scores):")
print(df.groupby(['Subject', 'Student'])['Score'].sum(), "\n")

# 4. Group by Subject and show describe summary

print("Describe Summary per Subject:")
print(df.groupby('Subject')['Score'].describe(), "\n")
```

Output:

```
Mean Score per Subject:
Subject
English    88.0
Math       87.5
Science    87.5

Multiple Aggregations per Subject:
      mean max min count
Subject
English  88.0  88  88     1
Math     87.5  90  85     2
Science  87.5  95  80     2

Group by Subject and Student (sum of scores):
Subject Student
English E      88
Math    A      90
```

DATA SCIENCE WITH PYTHON

```
B      85
Science C    95
D      80

Describe Summary per Subject:

count mean  std  min  25%  50%  75%  max
Subject
English  1.0  88.0  NaN  88.0  88.0  88.0  88.0  88.0
Math     2.0  87.5  3.54  85.0  86.25  87.5  88.75  90.0
Science  2.0  87.5  10.61  80.0  83.75  87.5  91.25  95.0
```

Data Visualization

Data visualization is the graphical representation of information and data. By using visual elements like charts, graphs, and maps, data visualization tools provide an accessible way to see and understand trends, outliers, and patterns in data. Additionally, it provides an excellent way for employees or business owners to present data to non-technical audiences without confusion.

In the world of Big Data, data visualization tools and technologies are essential to analyse massive amounts of information and make data-driven decisions.

Key Aspects/Importance of Data Visualization in Data Science:

- **Transformation:** It translates massive, complex datasets into simplified visual elements like dashboards, infographics, and heatmaps.
- **Insight Discovery:** Helps data scientists quickly spot relationships, trends, and anomalies that would be hidden in spreadsheets.
- **Communication:** Enables clear presentation of findings, allowing stakeholders to grasp key messages and stories from data without deep technical knowledge.
- **Decision Making:** Fuels informed decisions by making data intuitive and actionable, improving efficiency and outcomes.

Types of Data Visualization

1. **Charts and Graphs:** They are used to visualize data, with charts comparing data points across categories or showing trends over time and graphs analyzing relationships between variables to identify correlations, trends and outliers. Examples: Bar Charts, Line Charts, Pie Charts, Scatter Plots, Histograms, Box Plots.

DATA SCIENCE WITH PYTHON

2. **Maps:** They are used to display geographical data which provides spatial context to trends and patterns. Examples: Geographic Maps, Heat Maps
3. **Dashboards:** They combine multiple visualizations into a single interface which provides real-time insights and interactive features for users to explore data.

Types of Data Visualization Charts:

- Basic Charts for Data visualization
 1. Bar chart
 2. Line chart
 3. Pie chart
 4. Scatter plots
 5. Histograms
- Advanced Charts for Data Visualization
 1. Heat map
 2. Area Chart
 3. Box plot (Box-and-Whisker Plot)
 4. Bubble Chart
 5. Tree Map
 6. Parallel Coordinates
 7. Choropleth Map
 8. Sankey Diagram
 9. Radar Chart (Spider Chart)
 10. Donut Chart
 11. Network Graph
 12. Gauge Chart
 13. Sunbath Chart
 14. Hexbin Plot
 15. Violin Plot
- Visualization Charts for Textual and Symbolic data
 1. Word Cloud
 2. Pictogram Chart
- Temporal and Trend Charts Data Visualization
 1. Stream Graph
 2. Gnatt Chart
 3. Waterfall Chart

• Data Visualization with Matplotlib:

Matplotlib is a Python library used for creating static, animated and interactive data visualizations. It is built on the top of NumPy and it can easily handle large datasets for creating various types of plots such as line charts, bar charts, scatter plots, etc.

DATA SCIENCE WITH PYTHON

- **Installing Matplotlib:** If Matplotlib is not already installed, use the following command:

```
""" pip install matplotlib """
```

- **Understanding the Plotting Workflow:** A visualization is built using four main components:

Data → Figure → Axes → plot

Figure: A **Figure** is the entire canvas or window that contains one or more plots. Think of it as the sheet of paper on which graphs are drawn.

Axes: An **Axes** object is the actual plotting area where charts are displayed. A single Figure can contain multiple Axes.

Plot: A plot is the visual representation of your data, such as a line, bar, scatter, or histogram.

A. Line Chart: Line chart is one of the basic plots and can be created using plot() function. It is used to represent a relationship between two data X and Y on a different axis.

Syntax:

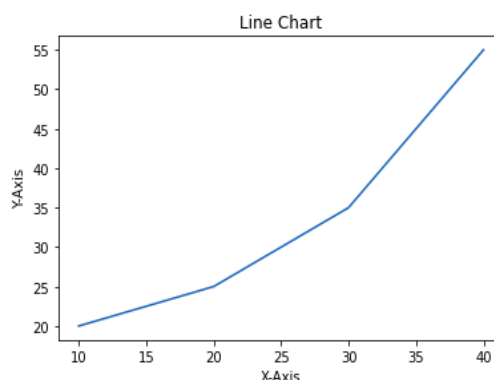
```
matplotlib.pyplot.plot(x, y)
```

Parameter: x, y Coordinates for data points.

Example: This code plots a simple line chart with labelled axes and a title using Matplotlib.

```
import matplotlib.pyplot as plt  
x = [10, 20, 30, 40]  
y = [20, 25, 35, 55]  
plt.plot(x, y)  
plt.title("Line Chart")  
plt.ylabel('Y-Axis')  
plt.xlabel('X-Axis')  
plt.show()
```

Output:



DATA SCIENCE WITH PYTHON

B. Bar Chart: Bar chart displays categorical data using rectangular bars whose lengths are proportional to the values they represent. It can be plotted vertically or horizontally to compare different categories.

Syntax:

```
matplotlib.pyplot.bar(x, height)
```

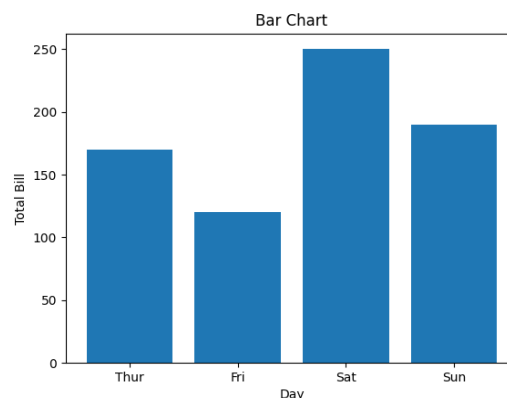
Parameter:

- x: Categories or positions on x-axis.
- height: Heights of the bars (y-axis values).

Example: This code creates a simple bar chart to show total bills for different days. X-axis represents the days and Y-axis shows total bill amount.

```
import matplotlib.pyplot as plt  
x = ['Thur', 'Fri', 'Sat', 'Sun']  
y = [170, 120, 250, 190]  
  
plt.bar(x, y)  
  
plt.title("Bar Chart")  
  
plt.xlabel("Day")  
  
plt.ylabel("Total Bill")  
  
plt.show()
```

Output:



C. Histogram: Histogram shows the distribution of data by grouping values into bins. The hist() function is used to create it, with X-axis showing bins and Y-axis showing frequencies.

Syntax:

```
matplotlib.pyplot.hist(x, bins=None)
```

DATA SCIENCE WITH PYTHON

Parameter:

- **x:** Input data.
- **bins:** Number of bins (intervals) to group data.

Example: This code plots a histogram to show frequency distribution of total bill values from the list **x**. It uses 10 bins and adds axis labels and a title for clarity.

```
import matplotlib.pyplot as plt

x = [7, 8, 9, 10, 10, 12, 12, 12, 13, 14, 14, 15, 16, 16, 17, 18, 18, 19, 20, 20,
     21, 22, 23, 24, 25, 25, 26, 28, 30, 32, 35, 36, 38, 40, 42, 44, 48, 50]

plt.hist(x, bins=10, color='steelblue')

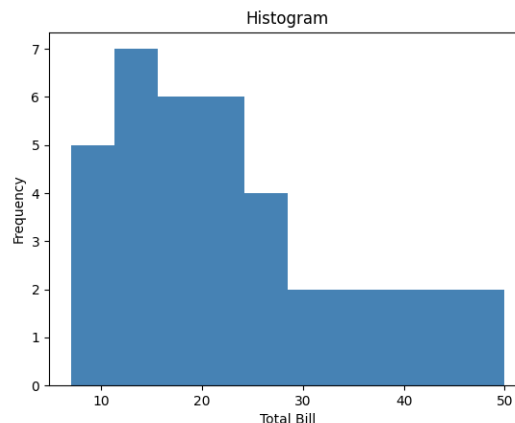
plt.title("Histogram")

plt.xlabel("Total Bill")

plt.ylabel("Frequency")

plt.show()
```

Output:



D. Scatter Plot: Scatter plots are used to observe relationships between variables. The `scatter()` method in the matplotlib library is used to draw a scatter plot.

Syntax:

```
matplotlib.pyplot.scatter(x, y)
```

Parameter: **x, y** Coordinates of the points.

Example: This code creates a scatter plot to visualize the relationship between days and total bill amounts using `scatter()`.

DATA SCIENCE WITH PYTHON

```
import matplotlib.pyplot as plt

x = ['Thur', 'Fri', 'Sat', 'Sun', 'Thur', 'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190, 160, 130, 240, 200]

plt.scatter(x, y)

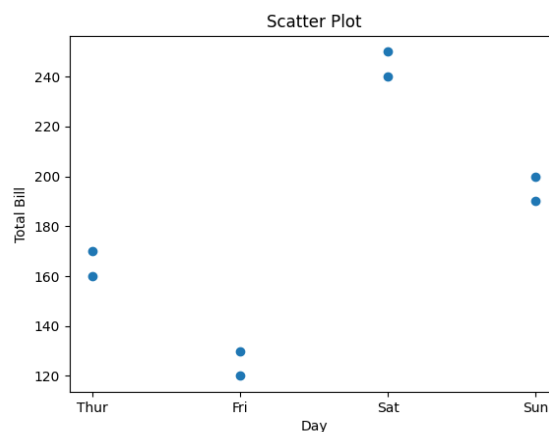
plt.title("Scatter Plot")

plt.xlabel("Day")

plt.ylabel("Total Bill")

plt.show()
```

Output:



E. Pie Chart: Pie chart is a circular chart used to show data as proportions or percentages. It is created using the `pie()`, where each slice (wedge) represents a part of the whole.

Syntax:

```
matplotlib.pyplot.pie(x, labels=None, autopct=None)
```

Parameter:

- `x`: Data values for pie slices.
- `labels`: Names for each slice.
- `autopct`: Format to display percentage (e.g., '%1.1f%%').

Example: This code creates a simple pie chart to visualize distribution of different car brands. Each slice of pie represents the proportion of cars for each brand in the dataset.

```
import matplotlib.pyplot as plt

import pandas as pd

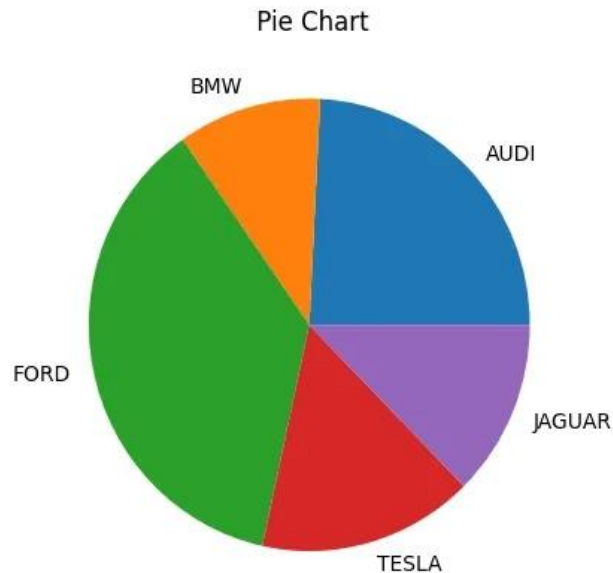
cars = ['AUDI', 'BMW', 'FORD', 'TESLA', 'JAGUAR',]

data = [23, 10, 35, 15, 12]
```

DATA SCIENCE WITH PYTHON

```
plt.pie(data, labels=cars)
plt.title(" Pie Chart")
plt.show()
```

Output:



F. Box plot: Box plot is a simple graph that shows how data is spread out. It displays the minimum, maximum, median and quartiles and also helps to spot outliers easily.

Syntax:

```
matplotlib.pyplot.boxplot(x, notch=False, vert=True)
```

Parameter:

- **x:** Data for which box plot is to be drawn (usually a list or array).
- **notch:** If True, draws a notch to show the confidence interval around the median.
- **vert:** If True, boxes are vertical. If False, they are horizontal.

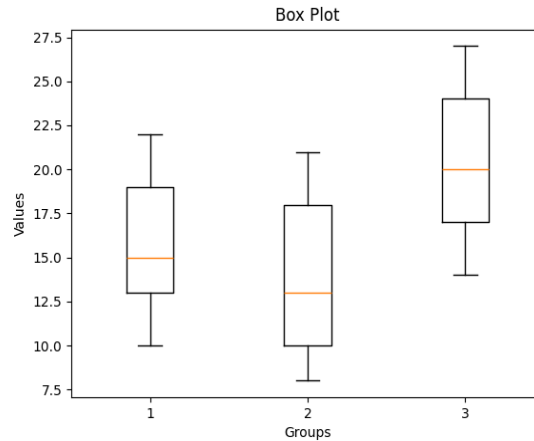
Example: This code creates a box plot to show the data distribution and compare three groups using matplotlib

```
import matplotlib.pyplot as plt
data = [ [10, 12, 14, 15, 18, 20, 22],
         [8, 9, 11, 13, 17, 19, 21],
         [14, 16, 18, 20, 23, 25, 27] ]
plt.boxplot(data)
plt.xlabel("Groups")
plt.ylabel("Values")
```

DATA SCIENCE WITH PYTHON

```
plt.title("Box Plot")  
plt.show()
```

Output:



G. Heatmap: Heatmap is a graphical representation of data where values are shown as colors. It helps visualize patterns, correlations or intensity in a matrix-like format. It is created using **imshow()** method in Matplotlib.

Syntax:

```
matplotlib.pyplot.imshow(X, cmap='viridis')
```

Parameter:

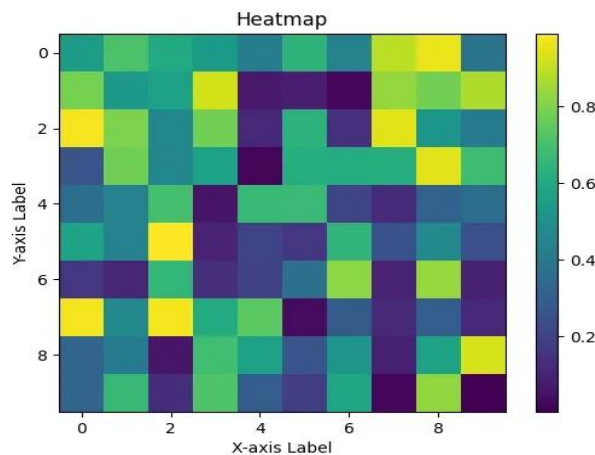
- **X:** 2D array (data to display as an image or heatmap).
- **cmap:** Sets the color map.

Example: This code creates a heatmap of random 10×10 data using `imshow()`. It uses 'viridis' color map and `colorbar()` adds a color scale.

```
import matplotlib.pyplot as plt  
import numpy as np  
np.random.seed(0)  
data = np.random.rand(10, 10)  
plt.imshow(data, cmap='viridis', interpolation='nearest')  
plt.colorbar()  
plt.xlabel('X-axis Label')  
plt.ylabel('Y-axis Label')  
plt.title('Heatmap')  
plt.show()
```

DATA SCIENCE WITH PYTHON

Output:



Explanation:

- `np.random.seed(0)`: Ensures same random values every time (reproducibility).
- `np.random.rand(10, 10)`: Generates a 10×10 array of random numbers between 0 and 1.

• Data Visualization with Seaborn:

Seaborn is a Python library for creating attractive statistical visualizations. Built on Matplotlib and integrated with Pandas, it simplifies complex plots like line charts, heatmaps and violin plots with minimal code.

A. Line Plot: A line plot shows the relationship between two numeric variables, often over time. It can also compare multiple groups using different lines.

Syntax:

```
sns.lineplot(x=None, y=None, data=None)
```

Parameters:

- `x, y`: Numeric input variables. These can be arrays, lists or column names from a DataFrame.
- `data`: DataFrame containing the data.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = {'Name': ['ANSH', 'SAHIL', 'JAYAN', 'ANURAG'], 'Age': [21, 23, 20, 24]}
df = pd.DataFrame(data)

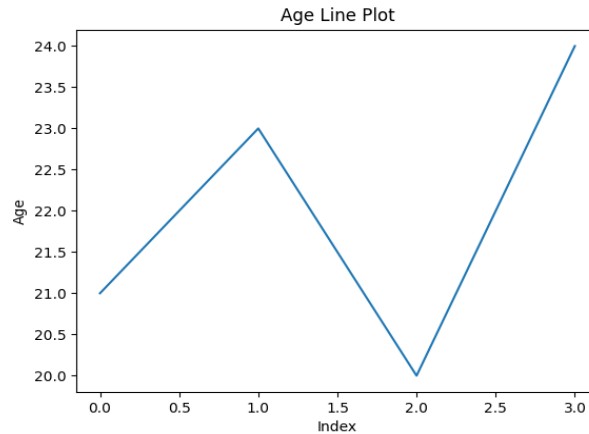
sns.lineplot(x=df.index, y='Age', data=df)

plt.xlabel('Index')
```

DATA SCIENCE WITH PYTHON

```
plt.ylabel('Age')  
plt.title('Age Line Plot')  
plt.show()
```

Output:



B. Scatter Plot: Scatter Plots are used to visualize the relationship between two numerical variables. They help identify correlations or patterns. It can draw a two-dimensional graph.

Syntax:

```
sns.scatterplot(x=None, y=None, data=None)
```

Parameters:

- x, y: Input data variables that should be numeric.
- data (optional): Dataset containing the variables.

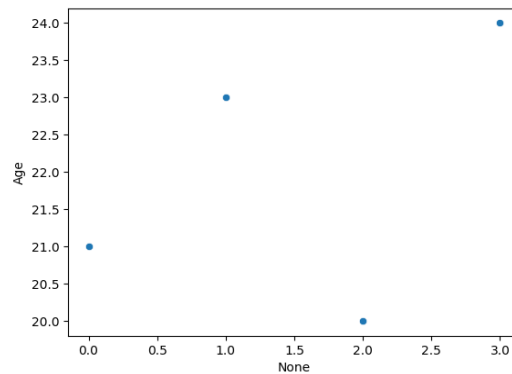
Returns: An Axes object with the scatter plot.

Example:

```
import pandas as pd  
import seaborn as sns  
import matplotlib.pyplot as plt  
data = {'Name': ['ANSH', 'SAHIL', 'JAYAN', 'ANURAG'], 'Age': [21, 23, 20, 24]}  
df = pd.DataFrame(data)  
sns.scatterplot(x=df.index, y='Age', data=df)  
plt.show()
```


DATA SCIENCE WITH PYTHON

Output:



C. Box plot: A Box plot is the visual representation of the depicting groups of numerical data with their quartiles against continuous/categorical data.

It consists of 5 key statistics: Minimum, First Quartile or 25%, Median (Second Quartile) or 50%, Third Quartile or 75% and Maximum

Syntax:

```
sns.boxplot(x=None, y=None, hue=None, data=None)
```

Parameters:

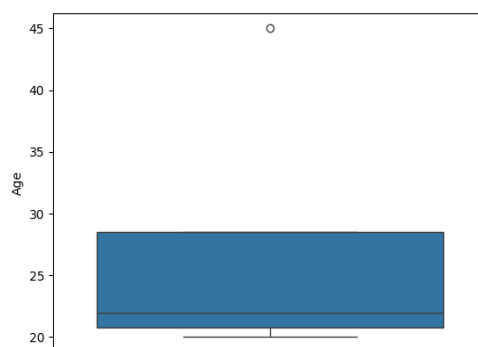
- x, y, hue: Variables for plotting long-form data.
- data: Dataset to plot. If x and y are absent data is treated as wide-form.

Returns: An Axes object with the box plot.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = {'Name': ['ANSH', 'SAHIL', 'JAYAN', 'ANURAG'], 'Age': [21, 23, 20, 45]}
df = pd.DataFrame(data)
sns.boxplot(y='Age', data=df)
plt.show()
```

Output:



DATA SCIENCE WITH PYTHON

D. Violin Plot: A Violin Plot is similar to a boxplot. It shows several quantitative data across one or more categorical variables such that those distributions can be compared.

Syntax:

```
sns.violinplot(x=None, y=None, hue=None, data=None)
```

Parameters:

- x, y, hue: Inputs for plotting long-form data.
- data: Dataset for plotting.

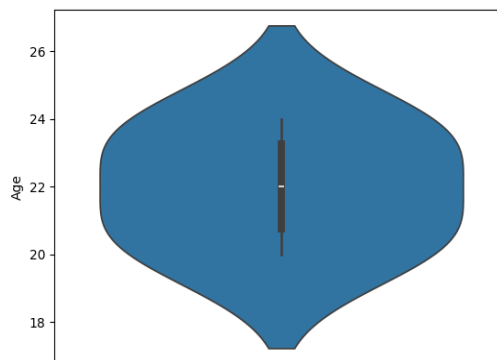
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {'Name': ['ANSH', 'SAHIL', 'JAYAN', 'ANURAG'], 'Age': [21, 23, 20, 24]}
df = pd.DataFrame(data)

sns.violinplot(y='Age', data=df)

plt.show()
```

Output:



E. Swarm Plot: A Swarm Plot displays individual data points without overlap along a categorical axis which provides a clear view of distribution density.

Syntax:

```
sns.swarmplot(x=None, y=None, hue=None, data=None)
```

Parameters:

- x, y, hue: Inputs for plotting long-form data.
- data: Dataset for plotting.

DATA SCIENCE WITH PYTHON

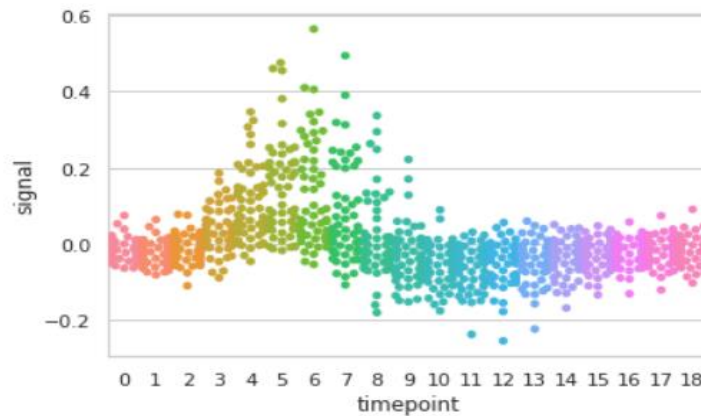
```
import seaborn

seaborn.set(style='whitegrid')

fmri = seaborn.load_dataset("fmri")    // "fmri" – is a random dataset

seaborn.swarmplot(x="timepoint",
                  y="signal",
                  data=fmri)
```

Output:



F. Point plot: Point Plot show point estimates and confidence intervals using scatter glyphs which represents the central tendency of a numeric variable.

Syntax:

```
sns.pointplot(x=None, y=None, hue=None, data=None)
```

Parameters:

- x, y: Inputs for plotting long-form data.
- hue: (optional) column name for color encoding.
- data: Dataframe as a Dataset for plotting.

Return: Axes object with the point plot.

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

data = {'Name': ['ANSH', 'SAHIL', 'JAYAN', 'ANURAG'], 'Age': [21, 23, 20, 24]}

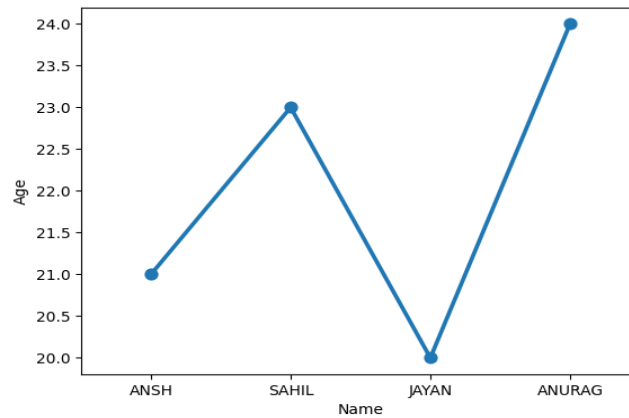
df = pd.DataFrame(data)

sns.pointplot(x='Name', y='Age', data=df)
```

DATA SCIENCE WITH PYTHON

```
plt.show()
```

Output:



G. Count plot: A Count Plot displays the number of occurrences of each category using bars to visualize the distribution of categorical variables.

Syntax :

```
sns.countplot(x=None, y=None, hue=None, data=None)
```

Parameters :

- x, y: Inputs for plotting long-form data.
- hue: (optional) column name for color encoding.
- data: Dataframe as a Dataset for plotting.

Returns: Axes object with the count plot.

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
data = {'Name': ['ANSH', 'SAHIL', 'ANSH', 'JAYAN', 'ANURAG', 'ANURAG', 'ANURAG', 'SAHIL']}
```

```
df = pd.DataFrame(data)
```

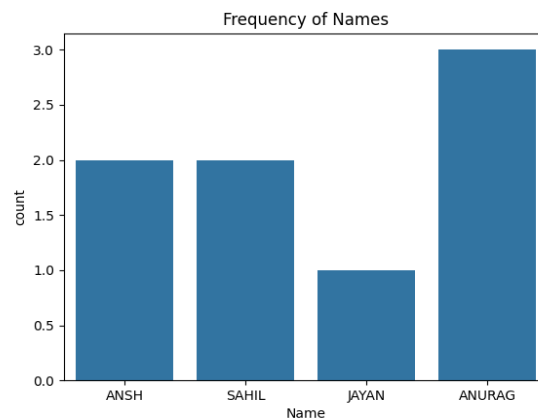
```
sns.countplot(x='Name', data=df)
```

```
plt.title("Frequency of Names")
```

```
plt.show()
```

DATA SCIENCE WITH PYTHON

Output:



H. KDE Plot: KDE Plot (Kernel Density Estimate) is used for visualizing the Probability Density of a continuous variable at different values in a continuous variable. We can also plot a single graph for multiple samples which helps in more efficient data visualization.

Syntax:

```
sns.kdeplot(data=None, x=None, y=None, fill=False)
```

Parameters:

- x, y: Vectors or data keys.
- vertical: Boolean to plot vertically.
- palette: Color palette.
- data: Dataframe

```
from sklearn.datasets import load_iris
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['Species'] = iris.target

df['Species'] = df['Species'].map({0: 'Setosa', 1: 'Versicolor', 2: 'Virginica'})

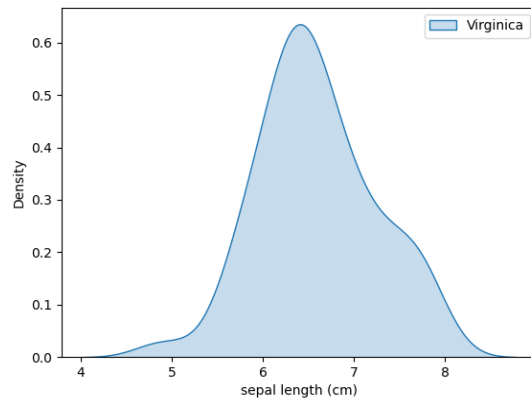
sns.kdeplot(data=df[df['Species'] == 'Virginica'], x='sepal length (cm)', fill=True,
label='Virginica')

plt.legend()

plt.show()
```

DATA SCIENCE WITH PYTHON

Output:



Model Building in Scikit-Learn

Scikit-learn is an open-source Python library that simplifies the process of building machine learning models. It offers a clean and consistent interface that helps both beginners and experienced users work efficiently.

- Supports tasks like classification, regression, clustering and preprocessing
- Makes model building fast and reliable
- Provides ready-to-use tools for training and evaluation
- Reduces complexity by avoiding manual implementation of algorithms

Installing and Using Scikit-learn

Before start building models, we need to install Scikit-learn. It requires Python 3.8 or newer and depends on two important libraries: NumPy and SciPy. Make sure these are installed first.

To install Scikit-learn run the following command:

```
pip install -U scikit-learn
```

This will download and install the latest version of Scikit-learn along with its dependencies.

Step 1: Loading a Dataset:

A dataset consists of:

- **Features (X):** Input variables that describe the data
- **Target (y):** The value we want to predict

Scikit-learn provides built-in datasets like Iris, Digits and Boston Housing. Using the Iris dataset:

- `load_iris()` loads the data
- X stores feature data

DATA SCIENCE WITH PYTHON

- y stores target labels
- feature_names and target_names give descriptive names

We can inspect the first few rows to understand the structure. For custom datasets, Pandas is commonly used to load external files such as CSVs.

```
from sklearn.datasets import load_iris

iris = load_iris()

X = iris.data

y = iris.target

feature_names = iris.feature_names

target_names = iris.target_names

print("Feature names:", feature_names)

print("Target names:", target_names)

print("\nType of X is:", type(X))

print("\nFirst 5 rows of X:\n", X[:5])
```

Output:

```
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target names: ['setosa' 'versicolor' 'virginica']

Type of X is: <class 'numpy.ndarray'>

First 5 rows of X:
[[5.1 3.5 1.4 0.2]
 [4.9 3. 1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5. 3.6 1.4 0.2]]
```

Step 2: Splitting the Dataset

To evaluate a model fairly, we split data into:

- Training set: Used to train the model
- Testing set: Used to evaluate how well the model generalizes

Using `train_test_split`, we split the Iris dataset so that 60% is for training and 40% for testing (`test_size=0.4`). `random_state=1` ensures reproducibility.

After splitting, we get:

- X_train, y_train -> Training data
- X_test, y_test -> Testing data

Checking the shapes ensures the data is split correctly.

DATA SCIENCE WITH PYTHON

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=1)
```

Now check the **Shapes** of the Splitted Data to ensures that both sets have correct proportions of data avoiding any potential errors in model evaluation or training.

```
print("X_train Shape:", X_train.shape)

print("X_test Shape:", X_test.shape)

print("Y_train Shape:", y_train.shape)

print("Y_test Shape:", y_test.shape)
```

Output:

```
X_train Shape: (90, 4)
X_test Shape: (60, 4)
Y_train Shape: (90,)
Y_test Shape: (60,)
```

Step 3: Handling Categorical Data

Machine learning algorithms work with numerical inputs, so categorical (text) data must be converted into numbers. If not encoded properly, models can misinterpret categories. Scikit-learn provides multiple encoding methods:

- 1. **Label Encoding:** It converts each category into a unique integer. For example in a column with categories like 'cat', 'dog' and 'bird', it would convert them to 0, 1 and 2 respectively. This method works well when the categories have a meaningful order such as “Low”, “Medium” and “High”.
- **LabelEncoder():** It is initialized to create an encoder object that will convert categorical values into numerical labels.
- **fit_transform():** This method first fits the encoder to the categorical data and then transforms the categories into corresponding numeric labels.

```
from sklearn.preprocessing import LabelEncoder

categorical_feature = ['cat', 'dog', 'dog', 'cat', 'bird']

encoder = LabelEncoder()

encoded_feature = encoder.fit_transform(categorical_feature)

print("Encoded feature:", encoded_feature)
```

Output:

```
Encoded feature: [1 2 2 1 0]
```


DATA SCIENCE WITH PYTHON

2. One-Hot Encoding: One-Hot Encoding creates separate binary columns for each category. This is useful when categories do not have any natural ordering. Example: cat, dog, bird -> 3 new columns (cat/dog/bird) with 1s and 0s.

- Input must be reshaped into a 2D array
- `OneHotEncoder(sparse_output=False)` generates binary columns

```
from sklearn.preprocessing import OneHotEncoder

import numpy as np

categorical_feature = ['cat', 'dog', 'dog', 'cat', 'bird']

categorical_feature = np.array(categorical_feature).reshape(-1, 1)

encoder = OneHotEncoder(sparse_output=False)

encoded_feature = encoder.fit_transform(categorical_feature)

print("OneHotEncoded feature:\n", encoded_feature)
```

Output:

```
OneHotEncoded feature:
[[0. 1. 0.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]]
```

Step 4: Training the Model

Now that our data is ready, it's time to train a machine learning model. Scikit-learn has many algorithms with a consistent interface for training, prediction and evaluation. Here we'll use Logistic regression as an example.

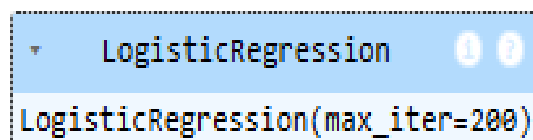
Note: We will not go into the details of how the algorithm works as we are interested in understanding its implementation only.

- **`log_reg = LogisticRegression(max_iter=200)`:** Creating a logistic regression classifier object.
- **`log_reg.fit(X_train, y_train)`:** Using this the logistic regression model adjusts the model's parameters to best fit the data.

```
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression(max_iter=200)

log_reg.fit(X_train, y_train)
```



```
LogisticRegression
LogisticRegression(max_iter=200)
```

DATA SCIENCE WITH PYTHON

Step 5: Make Predictions

Once trained we use the model to make predictions on the test data `X_test` by calling the `predict` method. These returns predicted labels `y_pred`.

- **log_reg.predict:** It uses trained logistic regression model to predict labels for the test data `X_test`.

```
y_pred = log_reg.predict(X_test)
```

Step 6: Evaluating Model Accuracy

Check how well our model is performing by comparing `y_test` and `y_pred`. Here we are using the `metrics` module's method `accuracy_score`.

```
from sklearn import metrics  
print("Logistic Regression model accuracy:", metrics.accuracy_score(y_test, y_pred))
```

Output:

```
Logistic Regression model accuracy: 0.9666666666666667
```

Now we want our model to make predictions on new sample data. Then the sample input can simply be passed in the same way as we pass any feature matrix. Here we used it as `sample = [[3, 5, 4, 2], [2, 3, 5, 4]]`

```
sample = [[3, 5, 4, 2], [2, 3, 5, 4]]  
preds = log_reg.predict(sample)  
pred_species = [iris.target_names[p] for p in preds]  
print("Predictions:", pred_species)
```

Output:

```
Predictions: [np.str_('virginica'), np.str_('virginica')]
```

Features of Scikit-learn

Scikit-learn is used because it makes building machine learning models straightforward and efficient. Here are some important reasons:

1. **Ready-to-Use Tools:** It provides built-in functions for common tasks like data preprocessing, training models and making predictions. This saves time by avoiding the need to code algorithms from scratch.
2. **Easy Model Evaluation:** With tools like cross-validation and performance metrics it helps to measure how well our model works and identify areas for improvement.
3. **Wide Algorithm Support:** It offers many popular machine learning algorithms including classification, regression and clustering which gives us flexibility to choose the right model for our problem.

DATA SCIENCE WITH PYTHON

4. **Smooth Integration:** Built on top of important Python libraries like NumPy and SciPy so it fits into our existing data analysis workflow.
5. **Simple and Consistent Interface:** The same straightforward syntax works across different models helps in making it easier to learn and switch between algorithms.
6. **Model Tuning Made Easy:** Tools like grid search help us fine-tune our model's settings to improve accuracy without extra hassle.

Benefits of using Scikit-learn

- **User-Friendly:** Scikit-learn's consistent and simple interface makes it accessible for beginners and best for experts.
- **Time-Saving:** Pre-built tools and algorithms reduce development time which allows us to focus more on solving problems than coding details.
- **Better Model Performance:** Easy-to-use tuning and evaluation tools help in improving model accuracy and reliability.
- **Flexible and Scalable:** Supports a wide range of algorithms and integrates smoothly with other Python libraries helps in making it suitable for projects of any size.
- **Strong Community Support:** A large, active community ensures regular updates, extensive documentation and plenty of resources to help when we get stuck.

Deep Learning with PyTorch and TensorFlow

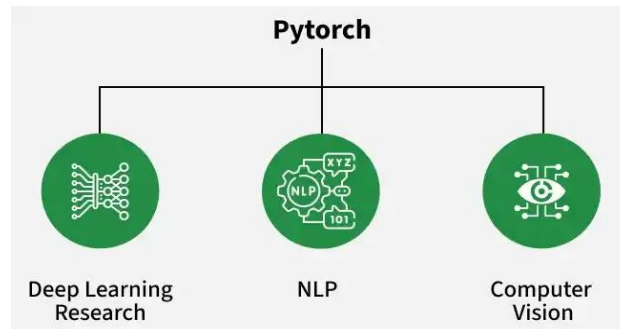
Deep Learning is a specialized branch of Machine Learning that enables computers to learn complex patterns from large amounts of data using artificial neural networks. Unlike traditional machine learning algorithms, deep learning models automatically extract meaningful features from raw data, making them highly effective for tasks such as image recognition, speech processing, natural language understanding, recommendation systems, and autonomous driving. With the increasing availability of powerful hardware and large datasets, deep learning has become one of the most influential technologies in modern Artificial Intelligence (AI).

- **Deep Learning with pytorch:**

PyTorch is a Python-based deep learning library that runs on CPU by default and supports GPU acceleration using CUDA (Compute Unified Device Architecture). It follows a define-by-run approach, creating dynamic computation graphs during execution, which makes debugging and customization easier.

DATA SCIENCE WITH PYTHON

- Uses dynamic graphs for flexibility
- Provides automatic differentiation for gradient computation
- Supports GPU acceleration with CUDA



- PyTorch can be installed on Windows, macOS and Linux using pip for CPU (without GPU):

`pip install torch torchvision torchaudio`

- Installing in notebooks (Jupyter Notebook or Google Colab):

`!pip install torch torchvision torchaudio`

A. PyTorch Tensors

Tensors are the fundamental data structures in PyTorch, similar to NumPy arrays but with GPU acceleration capabilities. PyTorch tensors support automatic differentiation, making them suitable for deep learning tasks.

```
import torch

x = torch.tensor([1.0, 2.0, 3.0])

print('1D Tensor: \n', x)

y = torch.zeros((3, 3))

print('2D Tensor: \n', y)
```

Output:

```
1D Tensor:
tensor([1., 2., 3.])
2D Tensor:
tensor([[0., 0., 0.],
        [0., 0., 0.],
        [0., 0., 0.]])
```

DATA SCIENCE WITH PYTHON

B. Operations on Tensors

```
a = torch.tensor([1.0, 2.0])
b = torch.tensor([3.0, 4.0])
print('Element Wise Addition of a & b: \n', a + b)
print('Matrix Multiplication of a & b: \n',
      torch.matmul(a.view(2, 1), b.view(1, 2)))
```

Output:

```
Element Wise Addition of a & b:
tensor([4., 6.])
Matrix Multiplication of a & b:
tensor([[3., 4.],
        [6., 8.]])
```

C. Reshaping and Transposing Tensors

Both `reshape()` and `view()` change the shape of a tensor without changing its data. However, `view()` requires the tensor to be stored contiguously in memory, while `reshape()` can handle more cases by creating a copy when necessary.

```
import torch
t = torch.tensor([[1, 2, 3, 4],
                  [5, 6, 7, 8],
                  [9, 10, 11, 12]])
print("Reshaping")
print(t.reshape(6, 2))
print("\nTransposing")
print(t.transpose(0, 1))
```

DATA SCIENCE WITH PYTHON

Output:

```
Reshaping
tensor([[ 1,  2],
        [ 3,  4],
        [ 5,  6],
        [ 7,  8],
        [ 9, 10],
        [11, 12]])

Resizing
tensor([[ 1,  2,  3,  4,  5,  6],
        [ 7,  8,  9, 10, 11, 12]])

Transposing
tensor([[ 1,  5,  9],
        [ 2,  6, 10],
        [ 3,  7, 11],
        [ 4,  8, 12]])
```

D. Autograd & Computational Graphs

The autograd module automates gradient calculation for backpropagation. This is important in training deep neural networks.

```
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2
y.backward()
print(x.grad)
```

Output:

```
tensor(4.)
```

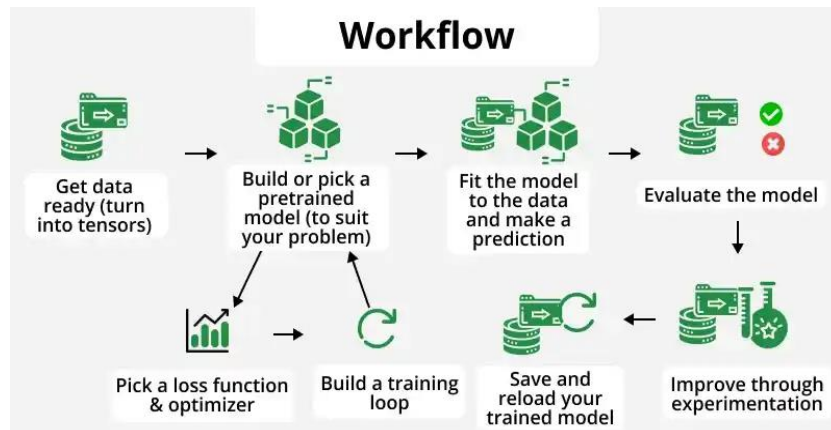
Explanation:

- `y = x ** 2` computes the square of `x` and records the operation in the computational graph.
- `y.backward()` performs backpropagation and calculates the gradient of `y` with respect to `x`.
- Since $y = x^2$, its derivative is $dy/dx = 2x$.
- For $x = 2$, the gradient is $2 \times 2 = 4$, which is stored in `x.grad`.

PyTorch dynamically creates a computational graph that tracks operations and gradients for backpropagation.

DATA SCIENCE WITH PYTHON

E. Building Neural Networks



In PyTorch, neural networks are built using the [torch.nn](https://pytorch.org/docs/stable/nn.html) module where:

- **nn.Linear(in_features, out_features)** defines a fully connected (dense) layer.
- Activation functions like `torch.relu`, `torch.sigmoid` or `torch.softmax` are applied between layers.
- **forward()** method defines how data moves through the network.

To build a neural network in PyTorch, create a class that inherits from `torch.nn.Module` and defines its layers and forward pass.

```
import torch

import torch.nn as nn

class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(10, 16)
        self.fc2 = nn.Linear(16, 8)
        self.fc3 = nn.Linear(8, 1)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = torch.sigmoid(self.fc3(x))
        return x

model = NeuralNetwork()
```

DATA SCIENCE WITH PYTHON

```
print(model)
```

Output:

```
NeuralNetwork(  
  (fc1): Linear(in_features=10, out_features=16, bias=True)  
  (fc2): Linear(in_features=16, out_features=8, bias=True)  
  (fc3): Linear(in_features=8, out_features=1, bias=True)  
)
```

E. Train the Model

The training involves generating dummy data (100 samples, each with 10 features). After this we run a training loop where we:

- `optimizer.zero_grad()` clears the accumulated gradients from the previous step.
- Forward Pass (`model(inputs)`) passes inputs through the model to generate predictions.
- Loss Computation (`criterion(outputs, targets)`) computes the difference between predictions and actual labels.
- Backpropagation (`loss.backward()`) computes gradients for all weights.
- Optimizer Step (`optimizer.step()`) updates the weights based on the computed gradients.

```
inputs = torch.randn((100, 10))  
targets = torch.randint(0, 2, (100, 1)).float()  
epochs = 20  
for epoch in range(epochs):  
    optimizer.zero_grad()  
    outputs = model(inputs)  
    loss = criterion(outputs, targets)  
    loss.backward()  
    optimizer.step()  
    if (epoch + 1) % 5 == 0:  
        print(f"Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}")
```


DATA SCIENCE WITH PYTHON

Output:

```
Epoch [5/20], Loss: 0.6727
Epoch [10/20], Loss: 0.6375
Epoch [15/20], Loss: 0.5925
Epoch [20/20], Loss: 0.5433
```

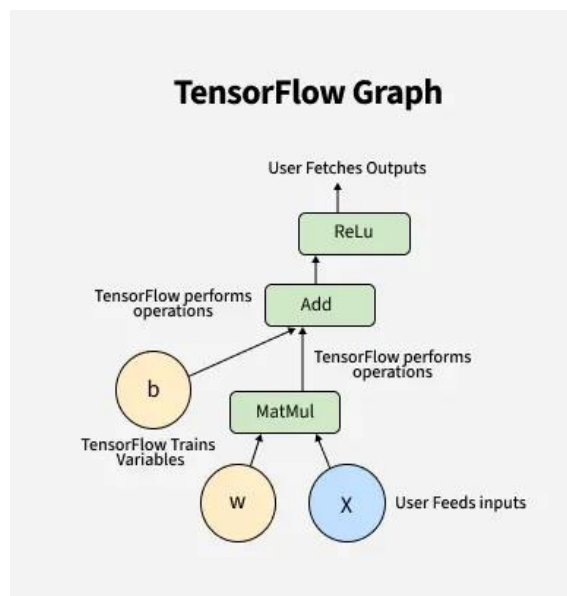
Applications

- i. Computer Vision:** Widely used in image classification, object detection and segmentation using CNNs and Transformers (e.g., ViT).
- ii. Natural Language Processing (NLP):** Supports transformers, recurrent neural networks (RNNs) and LSTMs for applications like text generation and sentiment analysis.
- iii. Reinforcement Learning:** Used in Deep Q-Networks (DQN), Policy Gradient Methods and Actor-Critic Algorithms.

- **Deep Learning with Tensorflow:**

TensorFlow is Google's open-source machine learning platform and the premier Python library used by data scientists to build, train, and deploy deep learning models.

- Supports a wide range of applications such as NLP, computer vision, time series forecasting and reinforcement learning
- Enables scalable model development and deployment across devices



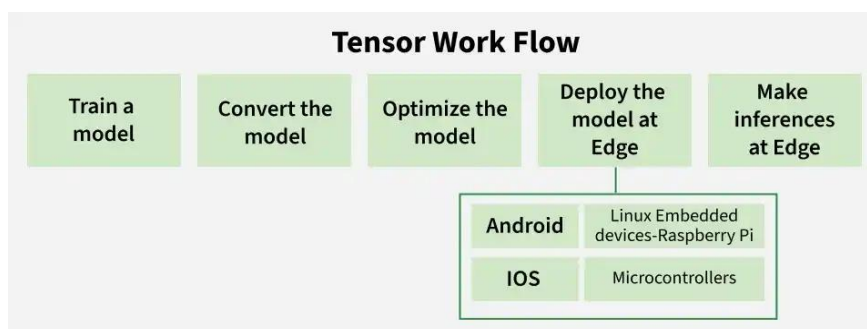
A. TensorFlow Architecture

The architecture of TensorFlow revolves around the concept of a computational graph which is a network of nodes (operations) and edges (data). Here's a breakdown of key components:

DATA SCIENCE WITH PYTHON

- **Tensors:** Tensors are the fundamental units of data in TensorFlow. They are multi-dimensional arrays or matrices used for storing data. A tensor can have one dimension (vector), two dimensions (matrix) or more dimensions.
- **Graph:** A TensorFlow graph represents a computation as a flow of tensors through a series of operations. Each operation in the graph performs a specific mathematical function on the input tensors such as matrix multiplication, addition or activation.
- In TensorFlow 2.x, computations are executed eagerly by default, meaning operations run immediately without requiring a separate session.

B. TensorFlow Workflow



C. Building a machine learning model in TensorFlow typically involves the following steps:

Step 1: Train a Model

- Use TensorFlow to build and train a machine learning model on platform like a PC or cloud.
- Employ datasets relevant to our application like images, text, sensor data, etc.
- Evaluate and validate the model to ensure high accuracy before deployment.

Step 2: Convert the Model

- Convert the trained model into TensorFlow Lite (.tflite) format using the TFLite Converter.
- This conversion prepares the model for resource-constrained edge environments.
- Supports different formats like saved models, Keras models or concrete functions.

Step 3: Optimize the Model

- Apply model optimization techniques such as quantization, pruning or weight clustering.
- Reduces the model size, improves inference speed and minimizes memory footprint.
- Crucial for running models efficiently on mobile, embedded or microcontroller devices.

DATA SCIENCE WITH PYTHON

Step 4: Deploy the Model

- Deploy the optimized .tflite model to edge devices like Android, iOS, Linux-based embedded systems like Raspberry Pi and Microcontrollers like Arm Cortex-M.
- Ensure compatibility with TensorFlow Lite runtime for the target platform.

Step 5: Make Inferences at the Edge

- Run real-time predictions directly on the edge device using the TFLite Interpreter.
- Enables low-latency, offline inference without relying on cloud computation.
- Supports use cases like image recognition, voice detection and sensor data analysis.

D. Comprehensive Ecosystem

TensorFlow offers a broad set of tools and libraries including:

- **TensorFlow Core:** The base API for TensorFlow that allows users to define models, build computations and execute them.
- **Keras:** [Keras](#) is integrated into TensorFlow (tf.keras) and is its official high-level API.
- **TensorFlow Lite:** A lightweight solution for deploying models on mobile and embedded devices.
- **TensorFlow.js:** A library for running machine learning models directly in the browser using JavaScript.
- **TensorFlow Extended (TFX):** A production-ready solution for deploying machine learning models in production environments.
- **TensorFlow Hub:** A repository of pre-trained models that can be easily integrated into applications.

E. Building a Simple Model with TensorFlow

Install tensorflow library by running the following command in command prompt

```
pip install tensorflow
```

Implementation

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(
```

DATA SCIENCE WITH PYTHON

```
(train_images.shape[0], 28 * 28)).astype('float32') / 255

test_images = test_images.reshape(
    (test_images.shape[0], 28 * 28)).astype('float32') / 255

model = Sequential([
    Dense(128, activation='relu', input_shape=(28 * 28,)),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.fit(train_images, train_labels, epochs=5)

test_loss, test_acc = model.evaluate(test_images, test_labels)

print(f"Test accuracy: {test_acc}")
```

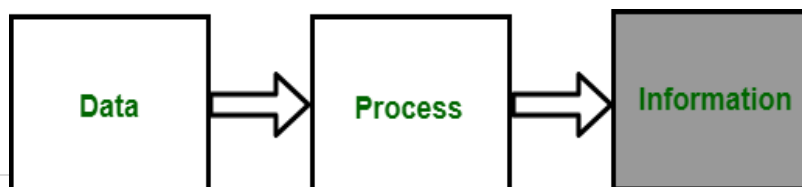
Output:

```
Epoch 1/5
1875/1875 ————— 7s 3ms/step - accuracy: 0.8771 - loss: 0.4387
Epoch 2/5
1875/1875 ————— 11s 3ms/step - accuracy: 0.9632 - loss: 0.1214
Epoch 3/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.9758 - loss: 0.0792
Epoch 4/5
1875/1875 ————— 7s 3ms/step - accuracy: 0.9829 - loss: 0.0563
Epoch 5/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.9869 - loss: 0.0442
313/313 ————— 1s 2ms/step - accuracy: 0.9727 - loss: 0.0869
Test accuracy: 0.9769999980926514
```

Data Preprocessing

- **Data Processing:**

Data processing means to processing of data i.e. to convert its format. As we all know data is the very useful and when it is well presented, and it becomes informative and useful. Data processing process system is also referred as **information system**. It is also right to say that data processing becomes the process of converting information into data.



DATA SCIENCE WITH PYTHON

A. Processing Data vs Processed Data

i. **Processing data definition** involves defining and managing the structure, characteristics, and specifications of data within an organization.

ii. **Processed data definition** typically refers to the refined and finalized specifications and attributes associated with data after it has undergone various processing steps.

Processing Data reflects the ongoing activities to improve data quality, while processed data represents the outcome of those activities—a refined and organized dataset that is poised for effective utilization. Both stages are integral to the data management lifecycle, contributing to the overall success of data-driven initiatives and informed decision-making within organizations.

In simple words, **processing of data** can be expressed as:

- Process of conversion of data in the computer understandable format.
- The sorting or processing of data by a computer.

B. Stages of Data Processing Process

Data processing process involves a series of stages to transform raw data into meaningful information. Here are the six fundamental stages of data processing process:

1. Collection

The process begins with the collection of raw data from various sources. The stage establishes the foundation for subsequent processing, ensuring a comprehensive pool of data relevant to the intended analysis. It could include surveys, sensors, databases, or any other means of gathering relevant information.

2. Preparation

Data preparation focuses on organizing, data cleaning, and formatting raw data. Irrelevant information is filtered out, errors are corrected, and the data is structured in a way that facilitates efficient analysis during subsequent stages of processing.

3. Input

During the data input stage, the prepared data is entered into a computer system. This can be achieved through manual entry or automated methods, depending on the nature of the data and the systems in place.

4. Data Processing

The core of data processing involves manipulating and analyzing the prepared data. Operations such as sorting, summarizing, calculating, and aggregating are performed to extract meaningful insights and patterns.

5. Data Output

DATA SCIENCE WITH PYTHON

The results of data processing are presented in a comprehensible format during the data output stage. This could include reports, charts, graphs, or other visual representations that facilitate understanding and decision-making based on the analyzed data.

6. Data Storage

The final stage requires storing the processed data for future reference and analysis. This is crucial for maintaining a historical record, enabling efficient retrieval, and supporting ongoing or future data-related initiatives. Proper data storage ensures the longevity and accessibility of valuable information.

C. Types of Data Processing

There are 7 types of Data processing:

1. Manual Data Processing

In this type, data is processed by humans without the use of machines or electronic devices. It involves tasks such as manual calculations, sorting, and recording, making it a time-consuming process.

2. Mechanical Data Processing

This type utilizes mechanical devices, such as punch cards or mechanical calculators, to process data. While more efficient than manual processing, it lacks the speed and capabilities of electronic methods.

3. Electronic Data Processing

Electronic Data Processing (EDP) involves the use of computers to process and analyze data. It significantly enhances speed and accuracy compared to manual and mechanical methods, making it a fundamental shift in data processing.

4. Batch Data Processing

Batch processing involves grouping data into batches and processing them together at a scheduled time. It is suitable for non-time-sensitive tasks and is efficient for large-scale data processing.

5. Real-time Data Processing

Real-time processing deals with data immediately as it is generated. It is crucial for time-sensitive applications, providing instant responses and updates, often seen in applications like financial transactions and monitoring systems.

6. Online Data Processing

Online Data Processing (OLTP) involves processing data directly while it is being collected. It is interactive and supports concurrent transactions, making it suitable for applications that require simultaneous user interaction and data updates.

DATA SCIENCE WITH PYTHON

7. Automatic Data Processing

Automatic Data Processing (ADP) refers to the use of computers and software to automate data processing tasks. It encompasses various methods, including batch processing and real-time processing, to efficiently handle large volumes of data with minimal human intervention.

D. Examples of Data Processing

- **Stock exchanges process** a massive volume of data during trades. The system matches buy and sell orders, updates stock prices in real-time, and records transaction details to maintain an accurate and transparent market.
- **In manufacturing**, data processing is used for quality control. Sensors and monitoring devices collect data on production processes, and algorithms analyze this information to identify defects, ensuring product quality.
- **Smart home devices** process data from sensors and user inputs to perform tasks like adjusting thermostats, controlling lights, and managing security systems based on predefined settings or user preferences.
- **Electronic Health Records (EHRs)** store and process patient data in healthcare settings. This includes medical history, test results, and treatment plans, facilitating efficient and accurate healthcare delivery.

- **Data Preprocessing:**

Data preprocessing is the first step in any data analysis or machine learning pipeline. It involves cleaning, transforming and organizing raw data to ensure it is accurate, consistent and ready for modeling. It has a big impact on model building such as:

- Clean and well-structured data allows models to learn meaningful patterns rather than noise.
- Properly processed data prevents misleading inputs, leading to more reliable predictions.
- Organized data makes it simpler to create useful inputs for the model, enhancing model performance.
- Organized data supports better Exploratory Data Analysis (EDA), making patterns and trends more interpretable.

Steps-by-Step implementation:

Step 1: Import Libraries and Load Dataset

Preparing the environment with libraries like pandas, numpy, scikit learn, matplotlib and seaborn for data manipulation, numerical operations, visualization and scaling. Load the dataset for preprocessing.

DATA SCIENCE WITH PYTHON

```
import pandas as pd

import numpy as np

from sklearn.preprocessing import MinMaxScaler, StandardScaler

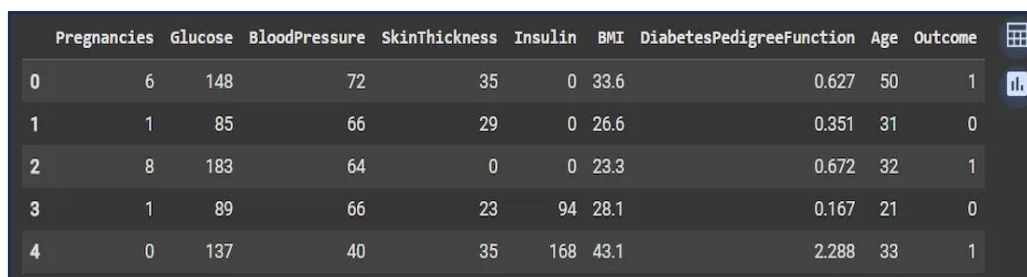
import seaborn as sns

import matplotlib.pyplot as plt

df = pd.read_csv('diabetes.csv')

df.head()
```

Output:



	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Step 2: Inspect Data Structure and Check Missing Values

We understand dataset size, data types and identify any incomplete (missing) data that needs handling.

- **df.info():** Prints concise summary including count of non-null entries and data type of each column.
- **df.isnull().sum():** Returns the number of missing values per column.

```
df.info()

print(df.isnull().sum())
```


DATA SCIENCE WITH PYTHON

Output:

```
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Pregnancies                          768 non-null    int64
1   Glucose                              768 non-null    int64
2   BloodPressure                       768 non-null    int64
3   SkinThickness                       768 non-null    int64
4   Insulin                             768 non-null    int64
5   BMI                                 768 non-null    float64
6   DiabetesPedigreeFunction            768 non-null    float64
7   Age                                 768 non-null    int64
8   Outcome                             768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

Step 3: Statistical Summary and Visualizing Outliers

Get numeric summaries like mean, median, min/max and detect unusual points (outliers). Outliers can skew models if not handled.

- **df.describe():** Computes count, mean, std deviation, min/max and quartiles for numerical columns.
- **Boxplots:** Visualize spread and detect outliers using matplotlib's boxplot().

```
df.describe()

fig, axs = plt.subplots(len(df.columns), 1, figsize=(7, 18), dpi=95)

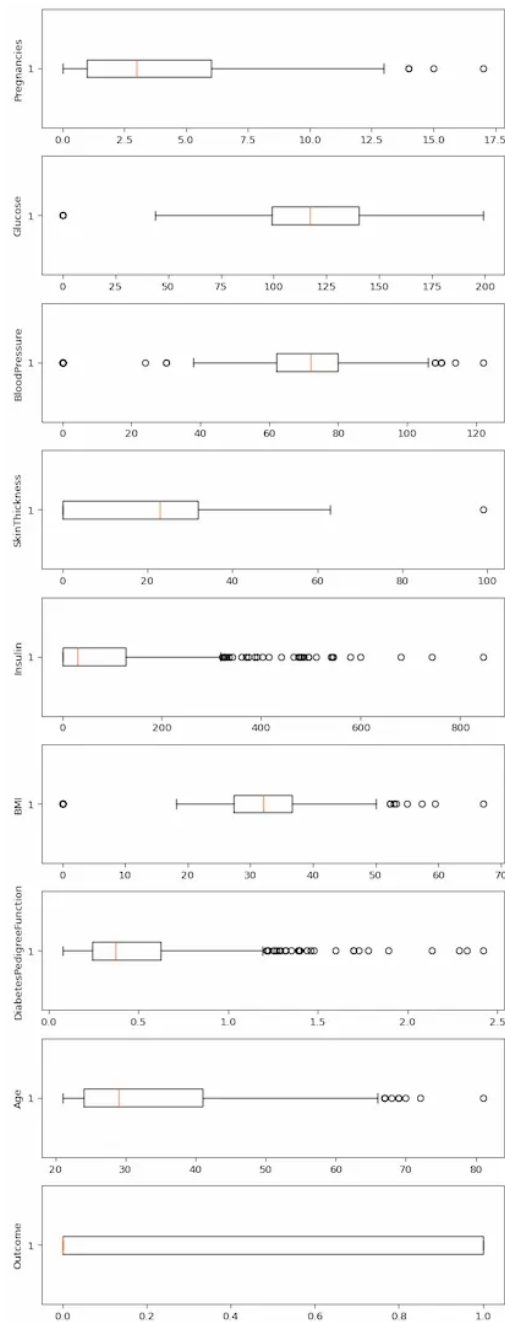
for i, col in enumerate(df.columns):
    axs[i].boxplot(df[col], vert=False)
    axs[i].set_ylabel(col)

plt.tight_layout()

plt.show()
```

DATA SCIENCE WITH PYTHON

Output:



Step 4: Remove Outliers Using the Interquartile Range (IQR) Method

Remove extreme values beyond a reasonable range to improve model robustness.

- $IQR = Q3 \text{ (75th percentile)} - Q1 \text{ (25th percentile)}$.
- Values below $Q1 - 1.5IQR$ or above $Q3 + 1.5IQR$ are outliers.
- Calculate lower and upper bounds for each column separately.
- Filter data points to keep only those within bounds.

DATA SCIENCE WITH PYTHON

```
q1, q3 = np.percentile(df['Insulin'], [25, 75])  
iqr = q3 - q1  
lower = q1 - 1.5 * iqr  
upper = q3 + 1.5 * iqr  
clean_df = df[(df['Insulin'] >= lower) & (df['Insulin'] <= upper)]
```

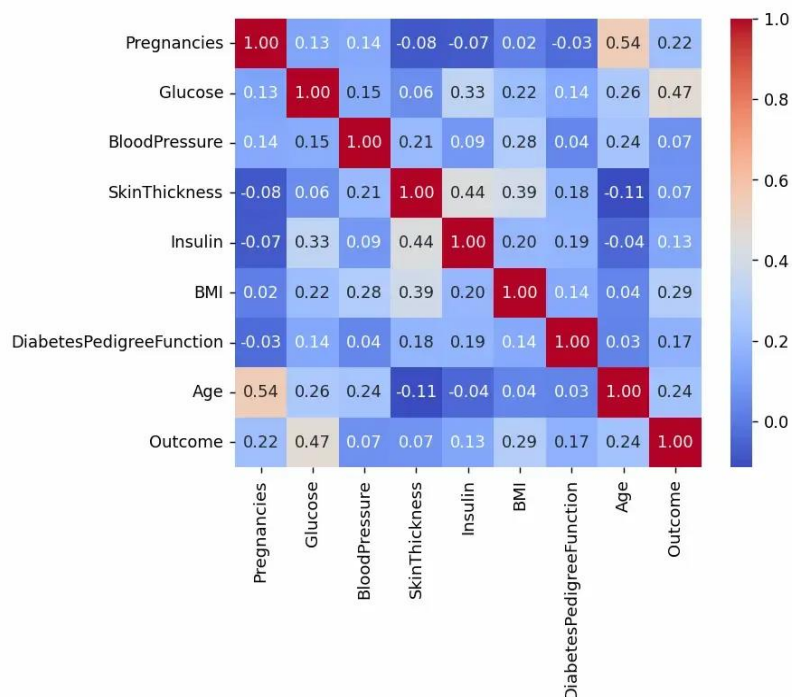
Step 5: Correlation Analysis

Understand relationships between features and the target variable (Outcome). Correlation helps gauge feature importance.

- **df.corr():** Computes pairwise correlation coefficients between columns.
- Heatmap via seaborn visualizes correlation matrix clearly.
- Sorting correlations with `corr['Outcome'].sort_values()` highlights features most correlated with the target.

```
corr = df.corr()  
plt.figure(dpi=130)  
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm')  
plt.show()  
print(corr['Outcome'].sort_values(ascending=False))
```

Output:



DATA SCIENCE WITH PYTHON

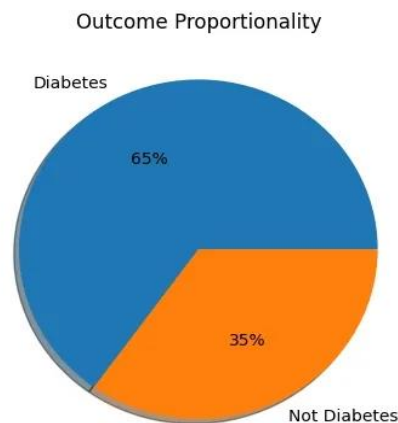
Step 6: Visualize Target Variable Distribution

Check if target classes (Diabetes vs Not Diabetes) are balanced, affecting model training and evaluation.

- **plt.pie():** Pie chart to display proportion of each class in the target variable 'Outcome'.

```
plt.pie(clean_df['Outcome'].value_counts(),
        labels=['Diabetes', 'Not Diabetes'],
        autopct='%f%%', shadow=True)
plt.title('Outcome Proportionality')
plt.show()
```

Output:



Step 7: Separate Features and Target Variable

Prepare independent variables (features) and dependent variable (target) separately for modeling.

- **df.drop(columns=[...]):** Drops the target column from features.
- Direct column selection `df['Outcome']` selects target column.

```
X = df.drop(columns=['Outcome'])
y = df['Outcome']
```

Step 8: Feature Scaling: Normalization and Standardization

Scale features to a common range or distribution, important for many ML algorithms sensitive to feature magnitudes.

1. Normalization (Min-Max Scaling): Rescales features between 0 and 1. Good for algorithms like k-NN and neural networks.

- **Class:** `MinMaxScaler` from `sklearn`.
- **.fit_transform():** Learns min/max from data and applies scaling.

DATA SCIENCE WITH PYTHON

```
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X)
print(X_normalized[:5])
```

Output:

```
array([[0.35294118, 0.74371859, 0.59016393, 0.35353535, 0.
        0.50074516, 0.23441503, 0.48333333],
       [0.05882353, 0.42713568, 0.54098361, 0.29292929, 0.
        0.39642325, 0.11656704, 0.16666667],
       [0.47058824, 0.91959799, 0.52459016, 0.
        0.34724292, 0.25362938, 0.18333333],
       [0.05882353, 0.44723618, 0.54098361, 0.23232323, 0.11111111,
        0.41877794, 0.03800171, 0.
        ],
       [0.
        0.68844221, 0.32786885, 0.35353535, 0.19858156,
        0.64232489, 0.94363792, 0.2
        ]])
```

2. Standardization: Transforms features to have mean = 0 and standard deviation = 1, useful for normally distributed features.

- **Class:** StandardScaler from sklearn.

```
scaler = StandardScaler()
X_standardized = scaler.fit_transform(X)
print(X_standardized[:5])
```

```
array([[ 0.63994726,  0.84832379,  0.14964075,  0.90726993, -0.69289057,
         0.20401277,  0.46849198,  1.4259954 ],
       [-0.84488505, -1.12339636, -0.16054575,  0.53090156, -0.69289057,
        -0.68442195, -0.36506078, -0.19067191],
       [ 1.23388019,  1.94372388, -0.26394125, -1.28821221, -0.69289057,
        -1.10325546,  0.60439732, -0.10558415],
       [-0.84488505, -0.99820778, -0.16054575,  0.15453319,  0.12330164,
        -0.49404308, -0.92076261, -1.04154944],
       [-1.14185152,  0.5040552 , -1.50468724,  0.90726993,  0.76583594,
         1.4097456 ,  5.4849091 , -0.0204964 ]])
```

DATA SCIENCE WITH PYTHON

- **Working with Missing Data:**

1. Using isnull()

isnull() returns a DataFrame of Boolean value where True represents missing data (NaN). This is simple if we want to find and fill missing data in a dataset.

Example 1: Finding Missing Values in a DataFrame

```
import pandas as pd

import numpy as np

d = {'First Score': [100, 90, np.nan, 95],
      'Second Score': [30, 45, 56, np.nan],
      'Third Score': [np.nan, 40, 80, 98]}

df = pd.DataFrame(d)

mv = df.isnull()

print(mv)
```

Output:

	First Score	Second Score	Third Score
0	False	False	True
1	False	False	False
2	True	False	False
3	False	True	False

Example 2: Filtering Data Based on Missing Values

Here used random Employee dataset. The isnull() function is used over the "Gender" column in order to filter and print out rows containing missing gender data.

```
import pandas as pd

d = pd.read_csv("/content/employees.csv")

bool_series = pd.isnull(d["Gender"])

missing_gender_data = d[bool_series]

print(missing_gender_data)
```

DATA SCIENCE WITH PYTHON

Output:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
0	Douglas	Male	8/6/1993	12:42 PM	97308	6.945	True	Marketing
1	Thomas	Male	3/31/1996	6:53 AM	61933	4.170	True	NaN
2	Maria	Female	4/23/1993	11:17 AM	130590	11.858	False	Finance
3	Jerry	Male	3/4/2005	1:00 PM	138705	9.340	True	Finance
4	Larry	Male	1/24/1998	4:47 PM	101004	1.389	True	Client Services
5	Dennis	Male	4/18/1987	1:35 AM	115163	10.125	False	Legal
6	Ruby	Female	8/17/1987	4:20 PM	65476	10.012	True	Product
7	NaN	Female	7/20/2015	10:43 AM	45906	11.598	NaN	Finance
8	Angela	Female	11/22/2005	6:29 AM	95570	18.523	True	Engineering
9	Frances	Female	8/8/2002	6:51 AM	139852	7.524	True	Business Development
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
994	George	Male	6/21/2013	5:47 PM	98874	4.479	True	Marketing
996	Phillip	Male	1/31/1984	6:30 AM	42392	19.675	False	Finance
997	Russell	Male	5/20/2013	12:39 PM	96914	1.421	False	Product
998	Larry	Male	4/20/2013	4:45 PM	60500	11.985	False	Business Development
999	Albert	Male	5/15/2012	6:24 PM	129949	10.169	True	Sales

855 rows × 8 columns

2. Using isna()

isna() returns a DataFrame of Boolean values where True indicates missing data (NaN). It is used to detect missing values just like isnull().

```
import pandas as pd

import numpy as np

data = {'Name': ['Amit', 'Sita', np.nan, 'Raj'],
        'Age': [25, np.nan, 22, 28]}

df = pd.DataFrame(data)

# Check for missing values using isna()

print(df.isna())
```

Output:

```
      Name  Age
0  False  False
1  False   True
2   True  False
3  False  False
```

DATA SCIENCE WITH PYTHON

3. Checking for Non-Missing Values Using notnull()

notnull() function returns a DataFrame with Boolean values where True indicates non-missing (valid) data. This function is useful when we want to focus only on the rows that have valid, non-missing values.

Example 1: Identifying Non-Missing Values in a DataFrame

```
import pandas as pd

import numpy as np

d = {'First Score': [100, 90, np.nan, 95],
      'Second Score': [30, 45, 56, np.nan],
      'Third Score': [np.nan, 40, 80, 98]}

df = pd.DataFrame(d)

nmv = df.notnull()

print(nmv)
```

Output:

	First Score	Second Score	Third Score
0	True	True	False
1	True	True	True
2	False	True	True
3	True	False	True

Example 2: Filtering Data with Non-Missing Values

notnull() function is used over the "Gender" column in order to filter and print out rows containing missing gender data.

```
import pandas as pd

d = pd.read_csv("/content/employees.csv")

nmg = pd.notnull(d["Gender"])

nmgd= d[nmg]

display(nmgd)
```


DATA SCIENCE WITH PYTHON

Output:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
0	Douglas	Male	8/6/1993	12:42 PM	97308	6.945	True	Marketing
1	Thomas	Male	3/31/1996	6:53 AM	61933	4.170	True	NaN
2	Maria	Female	4/23/1993	11:17 AM	130590	11.858	False	Finance
3	Jerry	Male	3/4/2005	1:00 PM	138705	9.340	True	Finance
4	Larry	Male	1/24/1998	4:47 PM	101004	1.389	True	Client Services
5	Dennis	Male	4/18/1987	1:35 AM	115163	10.125	False	Legal
6	Ruby	Female	8/17/1987	4:20 PM	65476	10.012	True	Product
7	NaN	Female	7/20/2015	10:43 AM	45906	11.598	NaN	Finance
8	Angela	Female	11/22/2005	6:29 AM	95570	18.523	True	Engineering
9	Frances	Female	8/8/2002	6:51 AM	139852	7.524	True	Business Development
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
994	George	Male	6/21/2013	5:47 PM	98874	4.479	True	Marketing
996	Phillip	Male	1/31/1984	6:30 AM	42392	19.675	False	Finance
997	Russell	Male	5/20/2013	12:39 PM	96914	1.421	False	Product
998	Larry	Male	4/20/2013	4:45 PM	60500	11.985	False	Business Development
999	Albert	Male	5/15/2012	6:24 PM	129949	10.169	True	Sales

855 rows × 8 columns

- **Filling Missing Values:**

1. Using `fillna()`

`fillna()` used to replace missing values (NaN) with a given value. Lets see various example for this.

Example 1: Fill Missing Values with Zero

```
import pandas as pd
import numpy as np

d = {'First Score': [100, 90, np.nan, 95],
     'Second Score': [30, 45, 56, np.nan],
     'Third Score': [np.nan, 40, 80, 98]}

df = pd.DataFrame(d)

df.fillna(0)
```

DATA SCIENCE WITH PYTHON

Output:

	First Score	Second Score	Third Score
0	100.0	30.0	0.0
1	90.0	45.0	40.0
2	0.0	56.0	80.0
3	95.0	0.0	98.0

2. Using replace()

Use `replace()` function to replace NaN values with a specific value.

```
import pandas as pd
import numpy as np
data = pd.read_csv("employees.csv")
data = data.replace(to_replace=np.nan, value=-99)
print(data[10:25])
```

Output:

10	Louise	Female	8/12/1980	9:01 AM	63241	15.132	True	-99
11	Julie	Female	10/26/1997	3:19 PM	102508	12.637	True	Legal
12	Brandon	Male	12/1/1980	1:08 AM	112807	17.492	True	Human Resources
13	Gary	Male	1/27/2008	11:40 PM	109831	5.831	False	Sales
14	Kimberly	Female	1/14/1999	7:13 AM	41426	14.543	True	Finance
15	Lillian	Female	6/5/2016	6:09 AM	59414	1.256	False	Product
16	Jeremy	Male	9/21/2010	5:56 AM	90370	7.369	False	Human Resources
17	Shawn	Male	12/7/1986	7:45 PM	111737	6.414	False	Product
18	Diana	Female	10/23/1981	10:27 AM	132940	19.082	False	Client Services
19	Donna	Female	7/22/2010	3:48 AM	81014	1.894	False	Product
20	Lois	-99	4/22/1995	7:18 PM	64714	4.934	True	Legal
21	Matthew	Male	9/5/1995	2:12 AM	100612	13.645	False	Marketing
22	Joshua	-99	3/8/2012	1:58 AM	90816	18.816	True	Client Services
23	-99	Male	6/14/2012	4:19 PM	125792	5.042	-99	-99
24	John	Male	7/1/1992	10:08 PM	97950	13.873	False	Client Services

- **Outlier detection:**

1. Z score for Outlier Detection

Outlier detection is an important task in data as identifying outliers can help us to understand the data better and improve the accuracy of our models. One common technique for detecting outliers is Z score. It is a statistical measurement that describes how far a data point is from the mean, expressed in

DATA SCIENCE WITH PYTHON

terms of **standard deviations**. It helps us to identify if a data point is relatively higher or lower than the mean and how far it deviates from the average value.

Z-Score Formula:

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- X = Value of the data point
- μ = Mean of the data
- σ = Standard deviation of the data

Interpretation of Z-Score

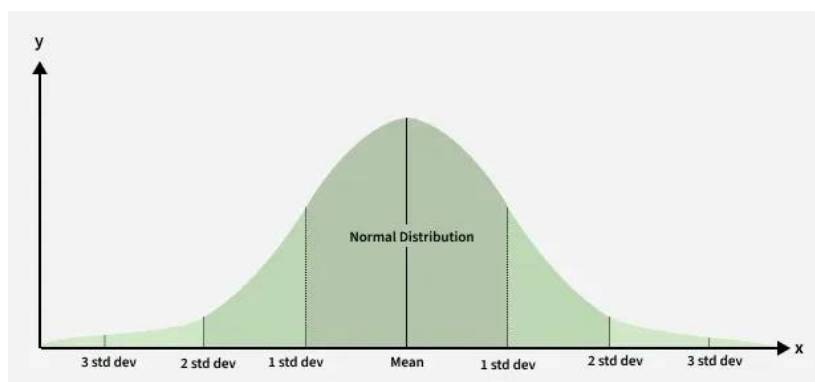
Outliers can be detected using Z-scores as follows:

- A **Z-score** of 0 shows that the data point is exactly at the mean.
- A **positive Z-score** shows that the data point is above the mean.
- A **negative Z-score** shows that the data point is below the mean.

Commonly, data points with a **Z-score greater than 3 or less than -3** are considered outliers as they lie more than 3 standard deviations away from the mean. This threshold can be adjusted based on the dataset and the specific needs of the analysis.

Why Use Z-Score for Outlier Detection?

Z-Score is useful for detecting outliers in Nominal Distribution, as it highlights data points that lie far from the mean. A normal distributions shown below and it is estimated that 68% of the data points lie between +/- 1 standard deviation. 95% of the data points lie between +/- 2 standard deviation 99.7% of the data points lie between +/- 3 standard deviation.



DATA SCIENCE WITH PYTHON

Steps for Z-Score Based Outlier Detection in Python

Step 1: Importing Necessary Libraries

numpy, pandas, scipy and matplotlib for calculating the Z-Score and visualizing the outliers.

```
import numpy as np
import pandas as pd
from scipy.stats import zscore
import matplotlib.pyplot as plt
```

Step 2: Creating the Dataset

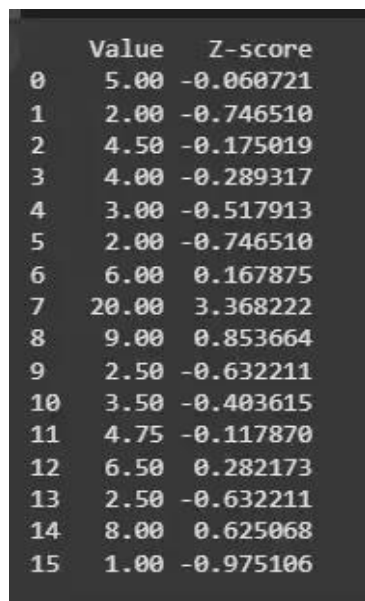
```
data = [5, 2, 4.5, 4, 3, 2, 6, 20, 9, 2.5, 3.5, 4.75, 6.5, 2.5, 8, 1]
```

```
df = pd.DataFrame(data, columns=['Value'])
```

Step 3: Calculating the Z-Scores

```
df['Z-score'] = zscore(df['Value'])
print(df)
```

Output:



	Value	Z-score
0	5.00	-0.060721
1	2.00	-0.746510
2	4.50	-0.175019
3	4.00	-0.289317
4	3.00	-0.517913
5	2.00	-0.746510
6	6.00	0.167875
7	20.00	3.368222
8	9.00	0.853664
9	2.50	-0.632211
10	3.50	-0.403615
11	4.75	-0.117870
12	6.50	0.282173
13	2.50	-0.632211
14	8.00	0.625068
15	1.00	-0.975106

Step 4: Identifying Outliers

Next, identify the data points that have a Z-score greater than 3 or less than -3 which are commonly considered outliers.

```
outliers = df[df['Z-score'].abs() > 3]
print(outliers)
```

DATA SCIENCE WITH PYTHON

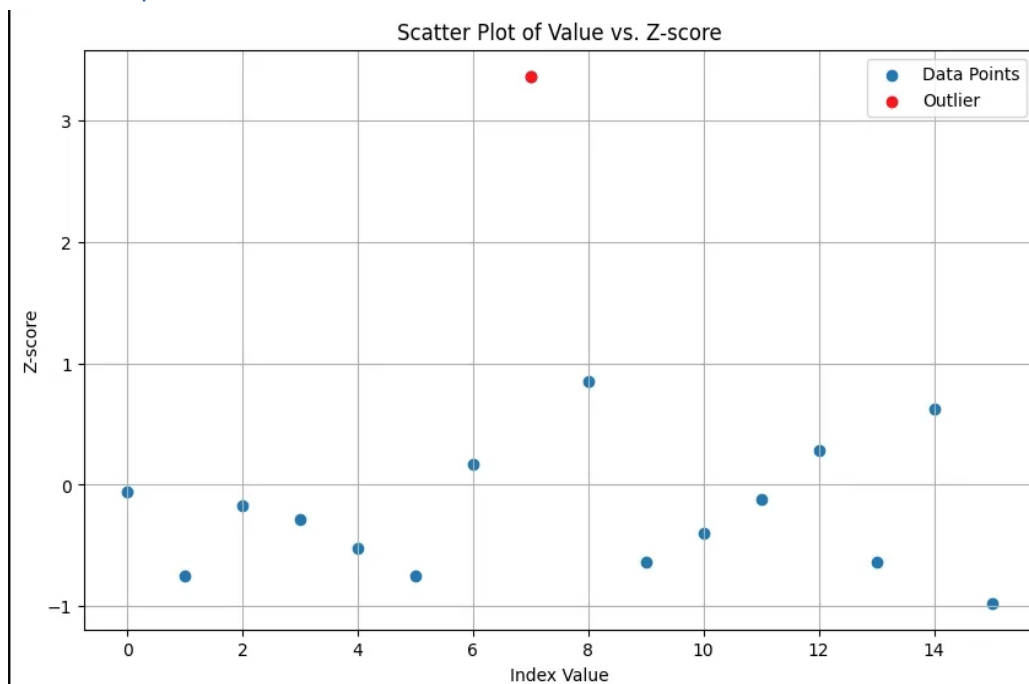
Output:

```
Value    Z-score
7    20.0    3.368222
```

Step 5: Visualizing the Data

```
plt.figure(figsize=(10, 6))
plt.scatter(df['Value'].index, df['Z-score'], label='Data Points')
plt.scatter(outliers['Value'].index, outliers['Z-score'], color='red', label='Outlier')
plt.xlabel('Index Value')
plt.ylabel('Z-score')
plt.title('Scatter Plot of Value vs. Z-score')
plt.legend()
plt.grid(True)
plt.show()
```

Output:



DATA SCIENCE WITH PYTHON

2. Interquartile Range to Detect Outliers in Data

IQR is used to **measure variability** by dividing a data set into quartiles. The data is sorted in ascending order and then we split it into 4 equal parts. The values Q1 (25th percentile), Q2 (50th percentile or median) and Q3 (75th percentile) separate dataset in 4 equal parts.

If a dataset has $2n$ or $2n+1$ data points, then

- Q2 = median of the dataset.
- Q1 = median of n smallest data points.
- Q3 = median of n highest data points.

The IQR is calculated as: **$IQR = Q3 - Q1$**

Data points that fall below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ are considered outliers.

Example:

Assume the below data:

6, 2, 1, 5, 4, 3, 50.

If these values represent the number of chapatis eaten in lunch then 50 is clearly an outlier. Use Python to detect it.

Step 1: Import necessary libraries.

```
import numpy as np
import seaborn as sns
```

Step 2: Sorting data in ascending order.

```
data = [6, 2, 3, 4, 5, 1, 50]
sort_data = np.sort(data)
sort_data
```

Output:

```
array([ 1, 2, 3, 4, 5, 6, 50])
```

Step 3: Calculating Q1, Q2, Q3 and IQR.

```
Q1 = np.percentile(data, 25, interpolation = 'midpoint')
Q2 = np.percentile(data, 50, interpolation = 'midpoint')
Q3 = np.percentile(data, 75, interpolation = 'midpoint')
print('Q1 25 percentile of the given data is, ', Q1)
print('Q1 50 percentile of the given data is, ', Q2)
```

DATA SCIENCE WITH PYTHON

```
print('Q1 75 percentile of the given data is, ', Q3)

IQR = Q3 - Q1

print('Interquartile range is', IQR)
```

Output:

```
Q1 25 percentile of the given data is, 2.5
Q1 50 percentile of the given data is, 4.0
Q1 75 percentile of the given data is, 5.5
Interquartile range is 3.0
```

Step 4: Find the lower and upper limits.

```
low_lim = Q1 - 1.5 * IQR

up_lim = Q3 + 1.5 * IQR

print('low_limit is', low_lim)

print('up_limit is', up_lim)
```

Output:

```
low_limit is -2.0
up_limit is 10.0
```

Step 5: Identify the outliers.

```
outlier = []

for x in data:

    if ((x > up_lim) or (x < low_lim)):

        outlier.append(x)

print(' outlier in the dataset is', outlier)
```

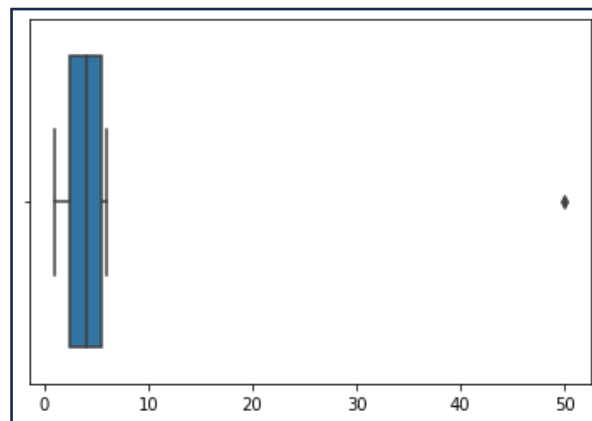
Output:

```
outlier in the dataset is [50]
```

Step 6: Plot the box plot to highlight outliers.

```
sns.boxplot(data)
```

DATA SCIENCE WITH PYTHON



DATA SCIENCE WITH PYTHON

Conclusion

Python has become one of the most influential programming languages in the field of data science because of its simplicity, flexibility, and extensive ecosystem of libraries. Throughout this handbook, we explored the fundamental concepts required to begin working with data using Python, including NumPy for numerical computing, Pandas for data manipulation, data visualization techniques, and essential data analysis workflows.

The objective of this handbook has been to provide a clear, practical, and beginner-friendly learning resource. Each topic has been explained with step-by-step examples to help readers understand both the theoretical concepts and their real-world applications. By practicing the examples and exercises presented in this handbook, readers will develop the confidence to work with datasets, perform exploratory data analysis, manipulate data efficiently, and build a strong foundation for more advanced topics such as machine learning, deep learning, artificial intelligence, and big data analytics.

Learning Python for data science is an ongoing journey. As new libraries, tools, and technologies continue to emerge, continuous practice and curiosity are essential for growth. Readers are encouraged to experiment with real-world datasets, contribute to open-source projects, and explore advanced frameworks to further enhance their skills.

DATA SCIENCE WITH PYTHON

Reference

- Harris, C.R. et al. "Array Programming with NumPy." NumPy Documentation. <https://numpy.org/doc/>
- The pandas development team. "Pandas Documentation." <https://pandas.pydata.org/docs/>
- Hunter, J.D. "Matplotlib: A 2D Graphics Environment." Matplotlib Documentation. <https://matplotlib.org/stable/>
- Waskom, M. "Seaborn: Statistical Data Visualization." Seaborn Documentation. <https://seaborn.pydata.org/>
- Pedregosa, F. et al. "Scikit-learn: Machine Learning in Python." Scikit-learn Documentation. <https://scikit-learn.org/stable/>
- Paszke, A. et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library." PyTorch Documentation. <https://pytorch.org/docs/>
- Abadi, M. et al. "TensorFlow: A System for Large-Scale Machine Learning." TensorFlow Documentation. https://www.tensorflow.org/api_docs
- Virtanen, P. et al. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python." SciPy Documentation. <https://docs.scipy.org/>
- GeeksforGeeks. "Data Analysis with Python." <https://www.geeksforgeeks.org/data-analysis/data-analysis-with-python/>
- GeeksforGeeks. "Python Data Visualization Tutorial." <https://www.geeksforgeeks.org/data-visualization/python-data-visualization-tutorial/>
- GeeksforGeeks. "Machine Learning." <https://www.geeksforgeeks.org/machine-learning/machine-learning/>
- GeeksforGeeks. "Introduction to Deep Learning." <https://www.geeksforgeeks.org/deep-learning/introduction-deep-learning/>
- GeeksforGeeks. "Image Processing in Python." <https://www.geeksforgeeks.org/python/image-processing-in-python/>
- GeeksforGeeks. "Computer Vision – Introduction." <https://www.geeksforgeeks.org/computer-vision/computer-vision-introduction/>
- GeeksforGeeks. "Introduction to Natural Language Processing." <https://www.geeksforgeeks.org/nlp/introduction-to-natural-language-processing/>
- GeeksforGeeks. "Python NumPy." <https://www.geeksforgeeks.org/numpy/python-numpy/>
- GeeksforGeeks. "Pandas DataFrame – loc, iloc, and Selection Methods." <https://www.geeksforgeeks.org/python/python-pandas-dataframe-loc/>
- GeeksforGeeks. "Scikit-learn – Model Building." <https://www.geeksforgeeks.org/machine-learning/learning-model-building-scikit-learn-python-machine-learning-library/>
- GeeksforGeeks. "ML – Understanding Data Processing." <https://www.geeksforgeeks.org/machine-learning/ml-understanding-data-processing/>
- GeeksforGeeks. "One Hot Encoding." <https://www.geeksforgeeks.org/machine-learning/ml-one-hot-encoding/>